



วิชาสหกิจศึกษา คณะวิทยาศาสตร์
มหาวิทยาลัยศรีนครินทรวิโรฒ

รายงานผลการปฏิบัติงานสหกิจศึกษา

โดย นางสาวสิริภัสสร ศรีวัฒนโณ รหัส 65102010424
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
คณะวิทยาศาสตร์

ปีการศึกษา 1/2568

รายงานผลการปฏิบัติงานสหกิจศึกษา

ณ บริษัทเคพีไอที เทค (ประเทศไทย) จำกัด

..... อาจารย์ที่ปรึกษาสหกิจศึกษา
(ผศ.ดร.นภา แซ่เป้)

..... อาจารย์ที่ปรึกษาสหกิจศึกษา
(อ.ดร.บรรพตธีร์ คมขำ)

..... Kamirapop Chino พนักงานที่ปรึกษา
(นายกวิณภาพ จิโน)

ชื่อรายงาน:	ระบบสร้างเอกสารตามความต้องการตามโมเดล Simulink Simulink Requirement Generator
ผู้จัดทำ:	นางสาวสิริภัสสร ศรีวัณโณ
สาขาวิชา/คณะ:	สาขาวิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยศรีนครินทรวิโรฒ
สถานที่ประกอบการ:	บริษัทเคพีไอที เทค (ประเทศไทย) จำกัด
อาจารย์ที่ปรึกษา:	ผศ.ดร.นภา แซ่เบ๊
อาจารย์นิเทศ:	ผศ.ดร.นภา แซ่เบ๊ อ.ดร.บรรพตริ คมขำ
ปีการศึกษา:	2568

บทคัดย่อ

(Abstract)

โครงการนี้มีวัตถุประสงค์เพื่อพัฒนาระบบจัดทำเอกสารข้อกำหนด (Requirement) สำหรับประกอบการทดสอบซอฟต์แวร์ในอุตสาหกรรมยานยนต์ โดยอัตโนมัติจากไฟล์ MATLAB Simulink (.slx) ซึ่งเดิมเป็นกระบวนการที่ใช้เวลานาน โครงการได้นำเทคโนโลยี Large Language Model (LLM) ที่บริษัทพัฒนาขึ้นมาประยุกต์ใช้ร่วมกับแนวทาง Prompt Engineer ระบบที่พัฒนาขึ้นประกอบด้วยส่วนติดต่อผู้ใช้แบบเว็บ (Web Application) ที่มีฟีเจอร์สำคัญ เช่น Chatbot สำหรับโต้ตอบด้วยภาษาธรรมชาติ ระบบจัดการไฟล์และโฟลเดอร์ การจัดการเครื่องมือ (Tool Management) และการส่งออกเอกสารในรูปแบบ Word และ PDF ผลลัพธ์ของโครงการแสดงให้เห็นถึงความสามารถในการลดเวลาในการจัดทำเอกสาร Software Requirement เพิ่มประสิทธิภาพการทำงานของทีม และสร้างต้นแบบสำหรับการขยายระบบเพื่อพัฒนาและต่อยอดต่อไปในอนาคต

คำสำคัญ: Simulink Model, SLX File (Matlab), Requirement Generator, Chatbot, Tool Management

กิตติกรรมประกาศ

(Acknowledgement)

การฝึกปฏิบัติสหกิจศึกษา ณ บริษัทเคพีไอที เทค (ประเทศไทย) จำกัด ครั้งนี้สำเร็จลุล่วงไปได้ดี ด้วยความกรุณาและการอนุเคราะห์อย่างยิ่งจาก ผศ.ดร.นภา แซ่เบ๊ และ อ.ดร.บรรพตริ คมขำ ซึ่งได้เสียเวลาอันมีค่าในการให้คำปรึกษา ให้ข้อคิดเห็น ความรู้และคำแนะนำต่างๆ ตลอดถึงการตรวจแก้รายงานให้ถูกต้อง ขอกราบขอบพระคุณอาจารย์ทั้งสองท่านเป็นอย่างสูงไว้ ณ ที่นี้

การจัดทำรายงานฉบับนี้ได้สำเร็จลุล่วงตามวัตถุประสงค์ได้ด้วยดี ผู้จัดทำต้องขอขอบพระคุณ บริษัทเคพีไอที เทค (ประเทศไทย) จำกัด คณะผู้บริหาร และแผนกทรัพยากรบุคคล ที่ให้โอกาสสำหรับการฝึกปฏิบัติสหกิจศึกษาในครั้งนี้ และสนับสนุนทรัพยากรที่จำเป็นสำหรับการดำเนินโครงการในระหว่างช่วงฝึกปฏิบัติสหกิจศึกษาในครั้งนี้ ขอขอบคุณ คุณกวิณภ จิโน ที่คอยให้คำแนะนำ ข้อมูลที่สำคัญ ตลอดจนการดูแลและสอนงานในด้านต่างๆ รวมถึงพี่ๆ ทีมงานและเพื่อนร่วมงานทุกๆ ท่านที่ให้ความช่วยเหลือ สนับสนุน และแลกเปลี่ยนความรู้ตลอดระยะเวลาการทำฝึกปฏิบัติสหกิจครั้งนี้ ขอขอบคุณเจ้าหน้าที่คณะวิทยาศาสตร์ ทุกท่านที่อำนวยความสะดวกในการปฏิบัติสหกิจศึกษา และบุคคลท่านอื่นๆ ที่ไม่ได้กล่าวนามทุกท่านที่ได้ช่วยส่งเสริมให้การฝึกปฏิบัติสหกิจศึกษาของข้าพเจ้าครั้งนี้ราบรื่นไปด้วยดี

ผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์และเป็นแนวทางที่ดีสำหรับผู้ที่มีความสนใจในการฝึกปฏิบัติงาน เพื่อเป็นการทราบถึงลักษณะงานเบื้องต้นของงานด้านการพัฒนาโปรแกรม ผู้จัดทำหากรายงานฉบับนี้มีข้อผิดพลาดประการใด ข้าพเจ้าก็ขออภัยมา ณ ที่นี้ด้วย

นางสาวสิริภัสสร ศรีวัฒนโณ

ผู้จัดทำรายงาน

สารบัญ

บทคัดย่อ.....	A
กิตติกรรมประกาศ	B
สารบัญ	C
1. บทนำ	1
1.1 ความเป็นมาและความสำคัญของโครงการ.....	1
1.2 รายละเอียดสถานประกอบการ	1
1.3 รายละเอียดและวัตถุประสงค์ของการปฏิบัติงานสหกิจศึกษา	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	3
2. ทฤษฎีที่เกี่ยวข้อง.....	4
2.1 ความรู้ที่ได้นำไปใช้หรือทฤษฎีที่เกี่ยวข้องกับโครงการหลัก	4
2.2 เครื่องมือและองค์ความรู้ที่ได้รับจากการปฏิบัติสหกิจ	7
3. วิธีดำเนินการ	17
3.1 แผนการดำเนินงาน	17
4. ผลปฏิบัติงาน.....	22
4.1 ผลงานโครงการหลัก.....	22
4.2 ผลการปฏิบัติงานประจำ	49
5. บทสรุป.....	51
5.1 สรุปผลและการนำไปใช้ประโยชน์.....	51
5.2 ปัญหาที่พบระหว่างการปฏิบัติงาน และการแก้ปัญหา	51
5.3 ข้อเสนอแนะ	54
6. บรรณานุกรม.....	56
ภาคผนวก	57
ประมวลผลภาพบรรยากาศการฝึกงาน	57

1. บทนำ

1.1 ความเป็นมาและความสำคัญของโครงการ

บริษัท เคพีไอที จำกัด (มหาชน) เป็นบริษัทระดับนานาชาติที่เชี่ยวชาญในการพัฒนาโซลูชันซอฟต์แวร์สำหรับอุตสาหกรรมยานยนต์ หนึ่งในกระบวนการสำคัญคือการรับไฟล์ MATLAB Simulink จากลูกค้าและนำมาจัดทำเอกสาร Software Requirement สำหรับประกอบการทดสอบระบบซอฟต์แวร์ ซึ่งกระบวนการนี้เดิมใช้เวลาค่อนข้างนานและมีปริมาณงานที่มาก เนื่องจากต้องจัดการทำเอกสารจำนวนมากด้วยวิธีปกติ

เพื่อแก้ปัญหานี้ ผู้จัดทำได้มาปฏิบัติงานตามโครงการสหกิจศึกษาในตำแหน่ง Software Developer Intern จึงริเริ่มโครงการพัฒนาระบบจัดทำ Software Requirement อัตโนมัติ โดยนำเทคโนโลยี LLM ที่บริษัทพัฒนาขึ้นมาประยุกต์ใช้ เพื่อเพิ่มความเร็ว และยังช่วยรักษาความปลอดภัยของข้อมูล พร้อมทั้งขยายขีดความสามารถของ LLM ให้รองรับสำหรับการใช้งานที่หลากหลายมากขึ้น

โครงการนี้มีความสำคัญเชิงกลยุทธ์ เนื่องจากช่วยลดเวลาในการพัฒนา เพิ่มประสิทธิภาพการทำงานของ ทีม รวมถึงสร้างความได้เปรียบในการแข่งขันในตลาดยานยนต์ที่ต้องการนวัตกรรมและความเร็วในการส่งมอบงาน ได้อย่างมีประสิทธิภาพ

1.2 รายละเอียดสถานประกอบการ

1. ชื่อ ที่อยู่ของสถานประกอบการ

ชื่อสถานประกอบการ: บริษัทเคพีไอที เทค (ประเทศไทย) จำกัด

ที่อยู่: 44/1 อาคารรุ่งโรจน์ธนกุล ชั้น 14 ถนนรัชดาภิเษก แขวงห้วยขวาง เขตห้วยขวาง

กรุงเทพมหานคร 10310

ธุรกิจหลักของบริษัท คือ การพัฒนาซอฟต์แวร์และการรวมตัวของอุตสาหกรรมขนส่ง

2. ชื่อ และ ตำแหน่งพนักงานที่ปรึกษา

ชื่อ นามสกุล: นายกวินภพ จิโน

ตำแหน่งในสถานประกอบการ: Associate Technical Specialist

1.3 รายละเอียดและวัตถุประสงค์ของการปฏิบัติงานสหกิจศึกษา

นิสิตปฏิบัติงานสหกิจศึกษา ในระยะเวลา 16 สัปดาห์

1.3.1 ตำแหน่งงาน: Software Developer Intern

ตารางที่ 1 แสดงตำแหน่งหน้าที่

หน้าที่หลัก	หน้าที่ย่อย
1. พัฒนาซอฟต์แวร์ระบบสร้าง Software Requirement	1. ร่วมกิจกรรมของทางบริษัท
2. พัฒนา Chatbot เพื่อเชื่อมกับระบบ Tool Management	2. จัดทำเอกสารเกี่ยวกับระบบ
3. ศึกษาค้นคว้าเพิ่มเติมเพื่อประกอบการทำโปรเจก	3. รายงานสถานะการทำงานกับพี่ Mentor
4. จัดทำการวัดผล และการทดสอบระบบ	4. อัปเดตโปรเจกตลง GitLab
	5. เข้าร่วม และ จัดทำกิจกรรม Knowledge Sharing

จากตารางที่ 1 แสดงรายละเอียดตำแหน่งงานและหน้าที่ความรับผิดชอบในระหว่างปฏิบัติสหกิจศึกษา โดยแบ่งเป็นหน้าที่หลัก ได้แก่ การพัฒนาระบบ สร้างเอกสาร Software Requirement และทำการพัฒนา Chatbot ที่สามารถรับข้อมูลไฟล์จากเว็บไซต์ของโปรเจกเว็บไซต์ HIL Commissioning มาประมวลผลร่วมกับเครื่องมือจากระบบต่างๆได้โดยสามารถจัดการตั้งค่าพื้นฐานผ่านทางระบบ Tool Management และระหว่างการศึกษาฝึกสหกิจนั้นได้มีการศึกษาค้นคว้าข้อมูลเพิ่มเติมทั้งจากข้อมูลของบริษัทเองและภายนอกเพื่อประกอบและอ้างอิงสำหรับการพัฒนาระบบ พร้อมทั้งมีการทำทดสอบระบบเบื้องต้นและจัดทำการวัดผลระบบที่มาจากใช้ LLM มาประกอบ และมีหน้าที่ย่อยคือ การเข้าร่วมกิจกรรมของทางบริษัทต่างๆ การจัดทำเอกสารของระบบเพื่อส่งต่อให้ทีมที่พัฒนาต่อไปสามารถนำไปต่อยอดต่อได้อย่างราบรื่น และระหว่างทำงานได้มีการรายงานความคืบหน้างานและอัปเดตโปรเจกตลง GitLab อย่างต่อเนื่อง พร้อมทั้งเข้าร่วมและจัดทำกิจกรรม Knowledge Sharing ทั้งหมดที่เกี่ยวข้อง

1.3.2 วัตถุประสงค์ของงานหลัก

1. เพื่อศึกษาทำความเข้าใจเกี่ยวกับเครื่องมือ AI ต่างๆที่มีอยู่ในบริษัท เพื่อรวบรวมข้อมูลสำคัญ และจัดทำเอกสารนำเสนอสำหรับแบ่งปันความรู้ และส่งเสริมความเข้าใจร่วมกันเกี่ยวกับการใช้งานและประโยชน์ของเครื่องมือเหล่านี้ภายในองค์กร รวมทั้งนำมาพัฒนาต่อยอดในระบบที่จะพัฒนา
2. เพื่อพัฒนาระบบให้สามารถช่วยลดระยะเวลาในการจัดทำ Software Requirement จากไฟล์ MATLAB (Simulink) โดยใช้ AI API ของทางบริษัทที่มีพัฒนาไว้ก่อนหน้านี้

3. เพื่อจัดทำและพัฒนาระบบ Tool Management เพื่อเชื่อมเข้ากับ Chatbot บนเว็บไซต์ HIL Commissioning ซึ่งมีส่วนสำหรับการจัดเก็บ Report และข้อมูลต่างๆเพื่อให้สามารถนำข้อมูลที่มีอยู่มาประยุกต์ใช้กับ LLM ได้มากขึ้น

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. สามารถเรียนรู้กระบวนการวิศวกรรมซอฟต์แวร์สำหรับพัฒนาระบบและนำไปปรับใช้ได้
2. การพัฒนาซอฟต์แวร์ในระดับองค์กรผ่านประสบการณ์และขั้นตอนการทำงานจริง
3. ได้เรียนรู้การพัฒนาระบบ จากการนำ API LLM มาประยุกต์ใช้ในระบบจริง

2. ทฤษฎีที่เกี่ยวข้อง

2.1 ความรู้ที่นำไปใช้หรือทฤษฎีที่เกี่ยวข้องกับโครงการหลัก

โครงการพัฒนาซอฟต์แวร์ที่ประสบความสำเร็จ ไม่ได้เกิดขึ้นจากเพียงบรรทัดของโค้ดเท่านั้น แต่ยังต้องอาศัยการประยุกต์ใช้หลักการทางทฤษฎีที่ถูกต้องและเหมาะสมตลอดกระบวนการพัฒนาอีกด้วย ตลอดช่วงระยะเวลาการฝึกงานครั้งนี้ การพัฒนาระบบ AI สำหรับสร้าง Software Requirements อัตโนมัติได้ยึดโยงอยู่กับแนวคิดสำคัญทั้งด้านวิทยาการคอมพิวเตอร์และวิศวกรรมซอฟต์แวร์ ซึ่งทำให้เห็นอย่างชัดเจนว่าความเข้าใจในหลักการเหล่านี้มีความจำเป็นอย่างยิ่งต่อการออกแบบสถาปัตยกรรมระบบและการแก้ปัญหาที่ซับซ้อนอย่างมีประสิทธิภาพ

2.2.1 VS Code

Visual Studio Code (VS Code) เป็นเครื่องมือพัฒนาโปรแกรมที่ได้รับความนิยมสูง เนื่องจากมีความยืดหยุ่นและรองรับการทำงานกับหลายภาษาโปรแกรม ในการปฏิบัติสหกิจ VS Code ถูกใช้เป็นเครื่องมือหลักในการเขียนและแก้ไขโค้ด โดยมีฟีเจอร์ที่ช่วยเพิ่มประสิทธิภาพ เช่น การจัดการโครงสร้างไฟล์ การใช้ส่วนขยาย (Extensions) เพื่อรองรับ Framework และการดีบั๊ก (Debugging) ทำให้การพัฒนาระบบมีความสะดวกและรวดเร็ว

2.2.2 Database

เพื่อรองรับฟีเจอร์ทั้งหมดของระบบ ฐานข้อมูลเป็นองค์ประกอบสำคัญในการจัดเก็บและบริหารข้อมูลอย่างมีประสิทธิภาพและเป็นสิ่งจำเป็น ไม่ว่าจะเป็นหลักการออกแบบและปรับแก้ Schema ของ Database ให้เหมาะสม ทั้งการออกแบบตาราง การสร้างความสัมพันธ์ และการจัดการข้อมูล เช่น sessions, ประวัติการแชต, รายละเอียดเครื่องมือ (tool definitions) และชุดคำสำคัญที่เกี่ยวข้อง ให้เป็นไปตามหลักการออกแบบ Normalization ซึ่งช่วยลดความซ้ำซ้อนของข้อมูลและเพิ่มความถูกต้อง ทำให้ระบบสามารถจัดการข้อมูลได้อย่างมีประสิทธิภาพและง่ายต่อการบำรุงรักษา

2.2.3 Full-Stack Web Development Architecture

แอปพลิเคชันนี้ถูกพัฒนาด้วยสถาปัตยกรรมแบบสามชั้น (Three-Tier Architecture) ซึ่งเป็นรูปแบบที่พิสูจน์แล้วว่ามีประสิทธิภาพและรองรับการแยกหน้าที่ของระบบได้ดี โครงสร้างนี้แบ่งเป็นเลเยอร์ของส่วนแสดงผล ตรรกะระบบ และชั้นข้อมูล ทำให้การแก้ไข UI ไม่ส่งผลกระทบต่อ Business Logic และสามารถปรับปรุงฐานข้อมูลได้โดยไม่ทำลายส่วนอื่นของระบบ

Frontend: ส่วนติดต่อผู้ใช้สร้างด้วย Svelte ซึ่งเป็นเฟรมเวิร์กแบบ Component-Based ที่รวดเร็วและมีประสิทธิภาพ

Backend: ใช้ Express (บน Node.js) ในการจัดการ API และเชื่อมกับ Python สำหรับกระบวนการประมวลผลไฟล์

Database: ใช้ MySQL เพื่อจัดเก็บข้อมูลถาวร ตั้งแต่ข้อมูล Session ของผู้ใช้งานถึง Metadata ของไฟล์ต่าง ๆ

2.1.4 REST API

REST API เป็นแนวทางการออกแบบบริการเว็บที่ใช้หลักการของ Representational State Transfer เพื่อให้การสื่อสารระหว่างระบบเป็นไปอย่างเรียบง่ายและมีมาตรฐาน ในโครงการนี้ REST API ถูกใช้ในการเชื่อมต่อระหว่างฝั่ง Frontend และ Backend ทำให้การแลกเปลี่ยนข้อมูลมีความชัดเจนและสามารถนำไปใช้งานได้มีประสิทธิภาพ

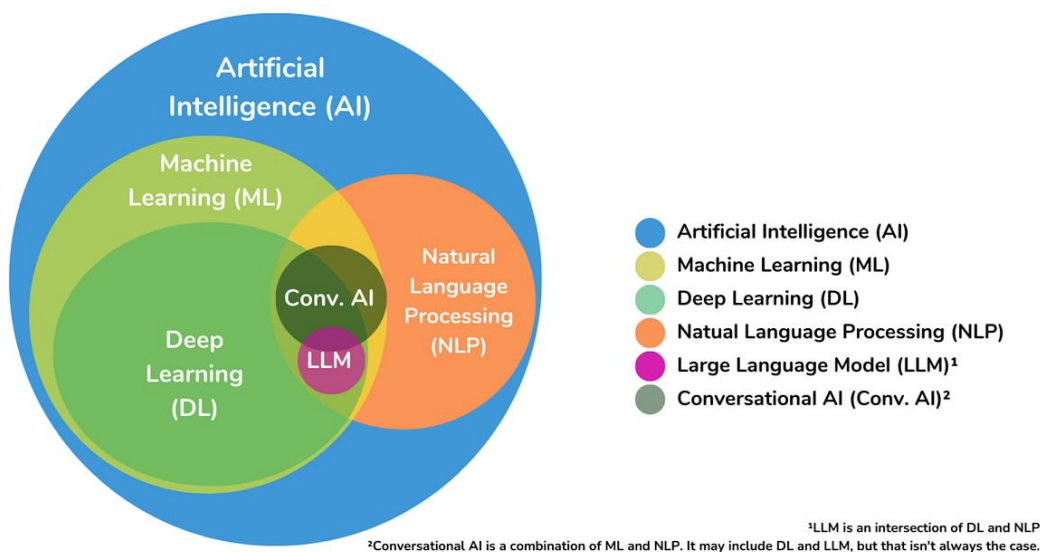
2.1.5 Data Structures

เพื่อจัดการความสัมพันธ์ที่ซับซ้อนของไฟล์ภายในโครงสร้างของ Simulink ฉันได้พัฒนาโครงสร้างข้อมูลแบบต้นไม้ (Tree Data Structure) เพื่อทำความเข้าใจลำดับชั้นของไฟล์และ dependencies ต่าง ๆ โครงสร้างนี้มีเมทอดสำหรับการไล่หา Node ในระดับ "leaf" และการไต่ย้อนกลับเพื่อหา "parent node" ซึ่งมีความสำคัญอย่างยิ่งต่อการวิเคราะห์บริบทของข้อมูล และยังเป็นโครงสร้างพื้นฐานของระบบจัดการไฟล์อีกด้วย

2.1.6 Git

Git เป็นระบบควบคุมเวอร์ชัน (Version Control System) ที่ช่วยจัดการการเปลี่ยนแปลงของโค้ดในโครงการอย่างมีประสิทธิภาพ โดยในระหว่างการพัฒนา Git ถูกใช้เพื่อบันทึกประวัติการแก้ไขไฟล์ ทำให้สามารถย้อนกลับไปยังเวอร์ชันก่อนหน้าได้ รวมถึงรองรับการทำงานร่วมกันในทีมผ่านการสร้าง Branch และการรวมโค้ด (Merge) ซึ่งช่วยลดความผิดพลาดและเพิ่มความเป็นระเบียบในการพัฒนา ในบริบทการพัฒนาพร้อมกับทีม

2.1.7 AI



ภาพที่ 1 แผนภาพแสดงความสัมพันธ์ AI และประเภท

จากภาพที่ 1 แผนภาพออยเลอร์นี้แสดงถึงความสัมพันธ์ของ AI (Artificial Intelligence), ML (Machine Learning), DL (Deep Learning), NLP (Natural Language Processing), LLM (Large Language Model) และ Conversational AI

2.1.7.1 AI (Artificial Intelligence)

หมายถึงการสร้างระบบที่สามารถทำงานเหมือนมี “ความฉลาด” เช่น การตัดสินใจ การแก้ปัญหา เป็นแนวคิดกว้าง ครอบคลุมทุกเทคนิคที่ทำให้เครื่องจักรคิดและเรียนรู้ได้

2.1.7.2 ML (Machine Learning)

เป็นส่วนหนึ่งของ AI ที่เน้นให้ระบบ เรียนรู้จากข้อมูล โดยไม่ต้องเขียนกฎแบบตายตัว ยกตัวอย่างเช่น การทำนายราคา และการจำแนกภาพ

2.1.7.3 DL (Deep Learning)

เป็นแขนงของ ML ที่ใช้ Neural Networks หลายชั้น เพื่อเรียนรู้ข้อมูลที่ซับซ้อน ยกตัวอย่างเช่น การรู้จำเสียง, การแปลภาษา, การสร้างภาพ

2.1.7.4 NLP (Natural Language Processing)

เป็นเทคนิคที่ทำให้คอมพิวเตอร์เข้าใจและประมวลผล ภาษามนุษย์ ยกตัวอย่างเช่นการแปลภาษา การสรุปข้อความ และการวิเคราะห์ความรู้สึก

2.1.7.5 LLM (Large Language Model)

เป็นโมเดล NLP ขนาดใหญ่ที่เรียนรู้จากข้อความจำนวนมาก ยกตัวอย่างเช่น GPT Claude และ LLaMA มีไว้ใช้สำหรับงานสร้างข้อความ, ตอบคำถาม, เขียนโค้ด

2.1.7.6 Conversational AI

ระบบ AI ที่ออกแบบมาเพื่อ สนทนาได้ตอบกับมนุษย์ ใช้ LLM + NLP + RAG เพื่อให้ตอบได้แม่นยำและมีบริบท ตัวอย่าง: Chatbot, Virtual Assistant

2.2 เครื่องมือและองค์ความรู้ที่ได้รับจากการปฏิบัติสหกิจ

การเชี่ยวชาญเครื่องมือสมัยใหม่เป็นสิ่งสำคัญต่อการพัฒนาซอฟต์แวร์อย่างมีประสิทธิภาพ การปฏิบัติสหกิจครั้งนี้เปิดโอกาสให้ฉันได้เรียนรู้ ฝึกฝน และพัฒนาการใช้ซอฟต์แวร์ เฟรมเวิร์ก และแพลตฟอร์มหลากหลายประเภท ซึ่งช่วยแปลงหลักการทางทฤษฎีให้กลายเป็นแอปพลิเคชันที่ใช้งานได้จริง และนอกเหนือจากทักษะด้านเทคนิคแล้ว ประสบการณ์ฝึกงานยังช่วยเสริมสร้างทักษะวิชาชีพที่สำคัญซึ่งจำเป็นต่อความสำเร็จในสภาพแวดล้อมองค์กรอีกด้วย

2.2.1 MVC Architecture

สถาปัตยกรรม MVC (Model-View-Controller) เป็นแนวทางการออกแบบซอฟต์แวร์ที่แยกส่วนการทำงานออกเป็นสามส่วน ได้แก่ Model สำหรับจัดการข้อมูล, View สำหรับแสดงผล และ Controller สำหรับควบคุมการทำงานของระบบ การใช้ MVC ช่วยให้โค้ดมีความเป็นระเบียบง่ายต่อการบำรุงรักษา และสามารถพัฒนาเพิ่มเติมได้โดยไม่กระทบต่อส่วนอื่นของระบบ

2.2.2 HIL

HIL (Hardware-in-the-Loop) คือเทคนิคการทดสอบระบบควบคุมหรือซอฟต์แวร์ที่เกี่ยวข้องกับฮาร์ดแวร์ โดยจำลองสภาพแวดล้อมจริงบางส่วนในรูปแบบเสมือน หรือ ซอฟต์แวร์จำลอง (Simulation) เพื่อแทนระบบจริง อย่างเช่น เครื่องยนต์, เซ็นเซอร์ และเชื่อมต่อกับฮาร์ดแวร์จริง เช่น ECU (Electronic Control Unit), Controller เพื่อทดสอบการทำงานผ่านการส่งสัญญาณเข้าออกเหมือนในสภาพแวดล้อมจริงก่อนนำไปใช้งานจริงในระบบเต็มรูปแบบ เช่น รถยนต์, เครื่องบิน, หรือระบบอุตสาหกรรม

2.2.3 GitLab

GitLab เป็นแพลตฟอร์มที่ใช้ร่วมกับ Git เพื่อจัดการโครงการซอฟต์แวร์ในรูปแบบออนไลน์ โดยมีฟีเจอร์สำคัญ เช่น การจัดเก็บ Repository การทำงานร่วมกันผ่าน Merge Request และการตั้งค่า CI/CD สำหรับการทดสอบและปรับใช้ระบบอัตโนมัติ ในโครงการสหกิจ GitLab ถูกใช้เพื่อควบคุมการทำงานของทีมงานและติดตามความคืบหน้าของโครงการ ทำให้การพัฒนาเป็นไปอย่างมีระบบ

2.2.4 SvelteKit

SvelteKit เป็นเฟรมเวิร์กสำหรับการพัฒนาเว็บแอปพลิเคชันที่เน้นความเร็วและประสิทธิภาพ โดยใช้แนวคิดการคอมไพล์โค้ดให้เป็น JavaScript ที่มีขนาดเล็กและทำงานได้รวดเร็ว ในโครงการนี้ SvelteKit ถูกนำมาใช้ในการสร้างส่วนติดต่อผู้ใช้ (UI) และจัดการการนำทาง (Routing) ทำให้ระบบมีประสบการณ์ใช้งานที่ราบรื่นและตอบสนองได้ดี

2.2.5 ExpressJS

ExpressJS เป็นเฟรมเวิร์กสำหรับการพัฒนาเว็บเซิร์ฟเวอร์บน Node.js ที่มีความยืดหยุ่นและใช้งานง่าย ในโครงการนี้ ExpressJS ถูกใช้ในการสร้าง API และจัดการคำขอ (Request) และคำตอบ (Response) ระหว่างฝั่งเซิร์ฟเวอร์และไคลเอนต์ ทำให้การสื่อสารข้อมูลมีความปลอดภัยและมีประสิทธิภาพ

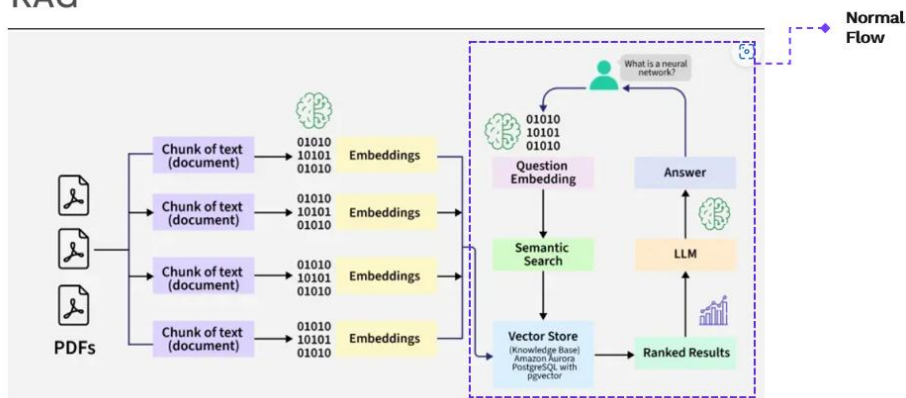
2.2.6 Bun

Bun เป็น JavaScript Runtime ที่มีความเร็วสูงและมาพร้อมเครื่องมือในตัว เช่น ตัวจัดการแพ็คเกจและตัวแปล TypeScript ในโครงการนี้ Bun ถูกนำมาใช้เพื่อรันแอปพลิเคชันและจัดการ Dependency ทำให้การพัฒนาและทดสอบระบบมีความรวดเร็วและลดความซับซ้อนในการตั้งค่า

2.2.7 Retrieval-Augmented Generation (RAG)

คือเทคนิคที่ใช้ในการปรับปรุงความสามารถของโมเดลภาษา (LLM) โดยการผสาน การค้นคืนข้อมูล (Retrieval) เข้ากับ การสร้างข้อความ (Generation) เพื่อให้โมเดลสามารถตอบคำถามได้แม่นยำขึ้น โดยอ้างอิงจากข้อมูลภายนอกที่เกี่ยวข้อง ไม่ใช่แค่ความรู้ที่ฝังอยู่ในโมเดลเท่านั้น

RAG



ภาพที่ 2 ภาพประกอบของ Retrieval-Augmented Generation (RAG)

จากรูปที่ 2 ขั้นตอนของ Retrieval-Augmented Generation (RAG) มีดังนี้

- Load (โหลดข้อมูล): ขั้นแรกคือการโหลดข้อมูลเข้ามาในระบบ โดยใช้ Document Loaders เพื่อดึงเอกสารหรือข้อมูลจากแหล่งต่าง ๆ เข้ามา
- Split (แยกข้อมูล): ใช้ Text Splitters ในการแบ่งเอกสารขนาดใหญ่ให้เป็นชิ้นเล็ก ๆ (chunks) เพื่อให้ง่ายต่อการจัดเก็บและค้นหา รวมถึงการส่งเข้าโมเดล เนื่องจากโมเดลมีข้อจำกัดเรื่องขนาดของ context window
- Store (จัดเก็บข้อมูล): หลังจากแยกข้อมูลแล้ว ต้องมีที่สำหรับจัดเก็บและทำดัชนี (index) เพื่อให้สามารถค้นหาได้ในภายหลัง โดยมักใช้ Vector Store ร่วมกับ Embeddings Model เพื่อแปลงข้อมูลเป็นเวกเตอร์สำหรับการค้นหาแบบ Semantic
- Retrieve (ดึงข้อมูล): เมื่อผู้ใช้ป้อนคำถาม ระบบจะค้นหาชิ้นข้อมูลที่เกี่ยวข้องจากที่จัดเก็บไว้ โดยใช้ Retriever
- Generate (สร้างคำตอบ): สุดท้าย Chat Model หรือ LLM จะสร้างคำตอบโดยใช้ prompt ที่รวมทั้งคำถามของผู้ใช้และข้อมูลที่ดึงมา

ซึ่งฟังก์ชันการทำงานหลักของ AI ในโครงการนี้อาศัยสถาปัตยกรรม RAG ซึ่งช่วยเสริมประสิทธิภาพของ Large Language Model (LLM) ด้วยข้อมูลภายนอกที่ดึงมาจากฐานความรู้ที่เกี่ยวข้อง

2.2.8 SoapUI

SoapUI เป็นเครื่องมือทดสอบ API ที่ได้รับความนิยมมาก โดยเฉพาะสำหรับ Web Services ที่ใช้ SOAP และ REST โดยจุดเด่นคือสามารถทำการทดสอบแบบ Functional, Load, และ Security ได้ในเครื่องมือเดียว ใช้งานง่ายโดยไม่ต้องเขียนโค้ดมาก

2.2.9 docx

เป็น Node.js library สำหรับสร้างไฟล์ .docx ตามมาตรฐาน Office Open XML ทำงานโดยสร้างโครงสร้าง XML ภายใน แล้วบีบอัดเป็น .docx โดยช่วยให้สามารถนำข้อความปกติมาสร้างและปรับแต่ง ข้อความตาราง Header/Footers Styles (ฟอนต์, ขนาด, สี) และ รูปภาพในไฟล์ .docx ได้

2.2.10 RACE Framework

RACE เป็นแนวทางการออกแบบ prompt สำหรับ AI ที่มีโครงสร้างชัดเจน โดยประกอบด้วย 4 องค์ประกอบหลัก ประกอบด้วย

ตารางที่ 2 แสดงถึง 4 องค์ประกอบหลักของ RACE Framework

องค์ประกอบ	คำอธิบาย	ตัวอย่าง
Role	กำหนดบทบาทหรือบุคลิกภาพของ AI เพื่อให้ AI ตอบสนองด้วยความเชี่ยวชาญเฉพาะด้าน	- นักวิเคราะห์ UX - นักการตลาดมืออาชีพ - วิศวกรผู้เชี่ยวชาญด้านยานยนต์
Action	ระบุสิ่งที่ต้องการให้ AI ทำ	- วิเคราะห์ข้อมูล - เขียนรายงาน - ประเมินกระบวนการ
Context	ให้ข้อมูลพื้นฐานหรือสถานการณ์ที่เกี่ยวข้อง	- กลุ่มเป้าหมาย - ปัญหาที่พบ - สภาพแวดล้อม
Expectation	ระบุรูปแบบหรือคุณภาพของผลลัพธ์ที่ต้องการ	- รายงานเชิงลึก - ข้อเสนอแนะที่นำไปใช้ได้จริง - สรุปแบบ bullet point

จากตารางที่ 2 บ่งบอกถึง 4 องค์ประกอบสำคัญของ RACE Framework ประกอบด้วย Role, Action, Context, และ Expectation ซึ่งสื่อถึงจุดเด่นของ RACE Framework คือ เน้นความเชี่ยวชาญเฉพาะด้าน ด้วยการกำหนด Role ที่ชัดเจน ช่วยลดความคลุมเครือ ด้วย Action ที่เฉพาะเจาะจง เพิ่มความแม่นยำ ด้วย Context ที่เกี่ยวข้องกับสถานการณ์จริง ควบคุมคุณภาพผลลัพธ์ ด้วย Expectation ที่ชัดเจน

เหมาะสำหรับนำมาใช้เมื่อ ต้องการคำตอบที่มีความเฉพาะเจาะจงและตรงตามบริบท ต้องการผลลัพธ์ที่มีโครงสร้างและคุณภาพสูง ใช้กับงานวิเคราะห์, วางแผน, สร้างเนื้อหา หรือประเมินสถานการณ์

2.2.11 Lang Chain

Lang Chain คือ Framework สำหรับการพัฒนาแอปพลิเคชันที่ใช้ Large Language Models (LLMs) เช่น GPT, Claude, หรือโมเดลอื่น ๆ โดยออกแบบมาเพื่อช่วยนักพัฒนา เชื่อมต่อโมเดลกับข้อมูลและเครื่องมือภายนอก ได้ง่ายขึ้น โดยมักถูกนำมาใช้พัฒนาร่วมกับภาษา JavaScript และ Python โดยมีข้อแตกต่างดังนี้

ตารางที่ 3 แสดงถึงความแตกต่างของ Lang Chain ระหว่างภาษา JavaScript และ Python

ฟีเจอร์	Python	JavaScript/TypeScript
LLM Integration	ครบ (OpenAI, Anthropic, HuggingFace, ฯลฯ)	ครบ (OpenAI, Anthropic, HuggingFace)
Prompt Templates	✓	✓
Chains	✓	✓
Agents	✓	ยังไม่ครบเท่า Python
Memory	✓	✓
Vector DB Support	ครบ (FAISS, Pinecone, Weaviate, Chroma)	รองรับ Pinecone, Weaviate, Supabase
Document Loaders	ครบมาก	รองรับพื้นฐาน (PDF, Text, Web)
Tools & Plugins	เยอะมาก	มี แต่ยังไม่ค่อยกว่า Python
Community Support	ใหญ่ที่สุด	กำลังโตเร็วมาก

จากตารางที่ 3 สามารถสรุปได้ว่า ใน Python นั้นมีฟีเจอร์ครบกว่า JavaScript เนื่องจากมีการพัฒนามาก่อนแต่ JavaScript ก็เริ่มเป็นที่นิยมในการนำมาใช้จึงมีการพัฒนาต่อเนื่องมาเรื่อยๆ JavaScript อาจจะขาดในส่วนของเครื่องมือในด้านต่างๆที่ยังไม่พัฒนามารองรับอย่างเช่น Vector Database บางประเภท FAISS และ Chroma ถ้าหากอยากใช้จะต้องไม่ใช้ framework และเขียนพัฒนาด้วยตนเอง

ความสามารถในการกำหนดค่าพารามิเตอร์ของ model ใน Lang chain ทั้งนี้ก็ขึ้นอยู่กับ Model นั้นๆ อีกด้วยโดยสามารถปรับค่าได้ดังนี้

- model ระบุชื่อโมเดล เช่น "gpt-4", "gpt-3.5-turbo", หรือโมเดลภายในองค์กร (kgpt-multimodal) โดยสามารถกำหนดว่าใช้โมเดลไหนสำหรับการตอบ

- temperature ควบคุมความสุ่มของคำตอบค่าน้อยคือเน้นความถูกต้อง ค่ามากคือคำตอบจะมีความหลากหลายและสร้างสรรค์มากขึ้น โดยค่ามักจะอยู่ในช่วง $[0,1]$ ขึ้นอยู่กับตัวชนิดของโมเดล
- max_tokens ไว้ใช้สำหรับจำกัดจำนวน token ในคำตอบ ป้องกันการตอบยาวเกินไป
- top_p (nucleus sampling) ควบคุมความหลากหลายของคำตอบ โดยเลือกจากความน่าจะเป็นรวม
- frequency_penalty / presence_penalty ปรับการซ้ำของคำในคำตอบ/ใช้ลดการตอบซ้ำหรือเพิ่มความหลากหลาย
- seed ทำให้ผลลัพธ์ reproducible (เหมือนเดิมทุกครั้ง)
- api_key / base_url สำหรับเชื่อมต่อ API ของ OpenAI หรือ proxy ภายในองค์กร
- streaming ถ้ามีค่าเป็น True จะส่งผลลัพธ์แบบ real-time (ส่งผลออกมาทีละ token)
- timeout กำหนดเวลารอคำตอบจาก API
- callbacks ใช้สำหรับ logging หรือ custom behavior ระหว่างการเรียกโมเดล

2.2.12 การประเมินผล

การวัดและประเมินผลจากโครงการนี้ มีตัวอย่างแค่เอกสาร Software Requirement ระบบเดียวและไม่ได้มีคำตอบที่ชัดเจน จึงไม่สามารถวัดเป็นค่า Accuracy / F1 Score ได้อย่างชัดเจน วิธีที่จะวัดผลได้ก็คือมี การวัดผลเชิงคุณภาพที่ใช้นักผู้ที่มีความเชี่ยวชาญมาตรวจสอบอีกที หรือวัดผลเชิงบริบทและความสามารถในการใช้ระบบมาช่วยตรวจสอบวัดผลคร่าวๆ จึงได้เน้นการศึกษาการวัดผลออกมาเป็นรูปแบบของ การวัดผลเชิงบริบทและความสามารถที่ไม่ต้องใช้นักผู้ที่มีรูปแบบการวัดผลตามรายการดังนี้

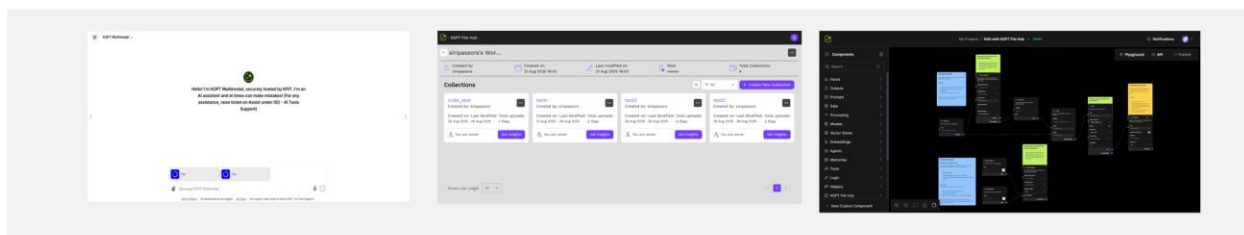
- Coverage: ตรวจสอบว่ารายงานครอบคลุมหัวข้อที่ควรกล่าวถึงครบถ้วนหรือไม่
- Structure: พิจารณาว่าโครงสร้างรายงานครบถ้วนและเรียงลำดับเนื้อหาอย่างถูกต้องหรือไม่
- Relevance: ตรวจสอบว่าเนื้อหาที่มีความเกี่ยวข้องกับสิ่งที่ได้รับมอบหมายให้เขียนหรือไม่
- Readability: ประเมินว่าภาษาที่ใช้ในรายงานอ่านเข้าใจง่ายหรือไม่ ยกตัวอย่างเช่นการทดสอบโดยใช้ Flesch Kincaid Grade Level
- Redundancy: ตรวจสอบว่ามีการกล่าวซ้ำหรือเนื้อหาที่ไม่จำเป็นมากเกินไปหรือไม่
- Units Consistency: ตรวจสอบว่าการใช้หน่วยตัวเลข เช่น km, THB, บาท มีความสม่ำเสมอหรือไม่

2.2.13 ระบบที่เกี่ยวข้องกับ LLM & AI ในบริษัท

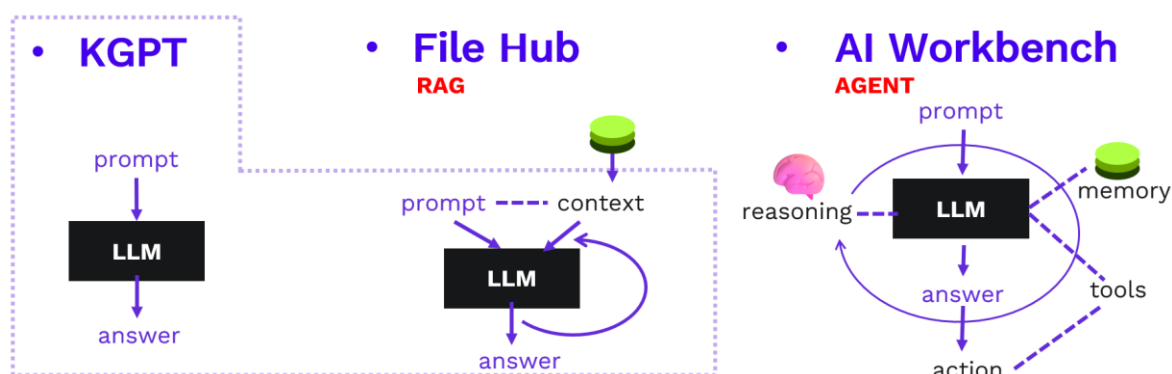
AI-related technologies currently used in KPIT.

Applications

- **KGPT**
similar to ChatGPT
- **File Hub**
similar to NoteBookLM
- **AI Workbench**
similar to n8n



ภาพที่ 3 ภาพของระบบที่เกี่ยวข้องกับ LLM & AI ในบริษัท



ภาพที่ 4 ภาพของกระบวนการทำงานของระบบที่เกี่ยวข้องกับ AI ในบริษัท

Application of KPIT			System	Have Reasoning	User Outer Information of LLM	Can access memory/tool/action
KGPT	FILE HUB	WORK BENCH				
			LLM	✓		
			RAG	✓	✓	
			Agent	✓	✓	✓

ภาพที่ 5 ภาพตารางเปรียบเทียบความสามารถของทั้งสามแอปพลิเคชัน

จากภาพที่ 3,4 และ 5 แสดงถึง 3 แอปพลิเคชันบนเว็บไซต์ที่เกี่ยวข้องกับ AI ของบริษัทที่พัฒนามาจากทางบริษัทเอง โดยมีฐานจากโมเดล llama ของ Facebook เพื่อให้เกิดความปลอดภัยทางด้านข้อมูลและเป็นการป้องกันไม่ให้ข้อมูลลั่วไหลไปสู่ นักพัฒนาโมเดลเจ้าอื่น ซึ่งขัดต่อนโยบายของบริษัท และนโยบายที่บริษัทมีต่อลูกค้า โดย KGPT เปลี่ยนเหมือน ChatGPT ที่มีการให้บริการทั้งในรูปแบบ API สำหรับนักพัฒนาและเว็บไซต์สำหรับพนักงานทั่วไป เป็นแอปพลิเคชันที่สามารถส่งคำถามไปให้ LLM Model และจะส่งผลลัพธ์ออกมาในรูปแบบคำตอบ ทำ Prompt template สำหรับใช้ผ่านหน้า UI และสามารถส่งไฟล์ขึ้นไปเพื่อเป็นข้อมูลเพิ่มเติ่มสำหรับตอบคำถามได้ แอปพลิเคชันถัดมาคือ File Hub ซึ่งมีทั้งเว็บไซต์สำหรับคนทั่วไป และ API สำหรับนักพัฒนา โดยมีความสามารถหลักที่โดดเด่นคือสามารถอัปโหลดไฟล์ขึ้นไป และมีหน้าแสดงผลสำหรับการจัดการไฟล์ผ่าน Workspace และ Collection ที่สามารถจำกัดสิทธิผู้ใช้งานได้ และมีระบบความปลอดภัยที่ได้มาตรฐาน และในส่วนสุดท้ายคือ AI Workbench ที่มีรูปแบบการทำงานเป็น AGENT ที่มีคุณสมบัติเหนือกว่าแอปพลิเคชัน KGPT ในส่วนที่สามารถรับ Tool อื่นๆเพื่อมาต่อเป็นกระบวนการอัตโนมัติ เช่น การเปิดเว็บไซต์เพื่อหาข้อมูล การเข้าเว็บไซต์แล้วจองตั๋วเครื่องบิน เป็นต้น ภายใน Workbench สามารถเชื่อม LLM Model จากภายนอกได้ทำให้มีความสามารถในการสร้างรูปภาพ กราฟ และอื่นๆเพิ่มเข้ามา แต่อาจจะไม่ได้ใช้เนื่องจากเหตุผลในด้านความปลอดภัยของข้อมูล

2.2.14 LLM Model ในบริษัท

ในบริษัทนั้นมี LLM Model ที่ทางบริษัทได้พัฒนาขึ้นมาและมีการอัปเดตเรื่อยๆ โดยก่อนการใช้งานต้องมีการ Raise Ticket เพื่อขอสิทธิการเข้าถึงแบ่งตามแต่ละโปรเจกต์ ในแต่ละงานควรมีการพิจารณาเลือกโมเดลให้เหมาะสมกับการใช้งานโดยพิจารณาจากคุณสมบัติของแต่ละโมเดล



KGPT API Updates

We're excited to introduce **kgpt-reasoning-text** with an expanded context length of **64k tokens** on the API!

 **Newly Available Models:**

- **kgpt-multimodal** (for multimodal tasks [image inputs]) – Context length 32k tokens.
- **kgpt-mini-text** (for simpler tasks) – Context length 16k tokens.
- **kgpt-text-embedding** (Nomic Embed Text compatible, same vector space) – For embedding tasks.
- **kgpt-reasoning-text** Optimized for complex tasks and supports a 64k token context length. Excellent for tool calling and large inputs.

 **Deprecation Notice:**

The following models will be removed:

- **internvl2-25b, internvl2.5-26b**: on September 15, 2025
- **llama3-70b, llama3.3-70b, gemma3-27b, phi-4, nomic-embed-text**: on September 30, 2025

 **Upgrade Recommendations:**

- Use **kgpt-multimodal** instead of **internvl2-25b/internvl2.5-26b/gemma3-27b** for multimodal tasks [image inputs].
- Use **kgpt-mini-text** instead of **phi-4** for simpler tasks.
- Use **kgpt-text-embedding** instead of **nomic-embed-text** for embedding tasks.
- Use **kgpt-reasoning-text** instead of **llama3-70b/llama3.3-70b** for complex tasks (supports tool calling).

ภาพที่ 6 LLM Model ที่มีอยู่ในบริษัท

จากภาพที่ 6 นี้แสดงถึงคุณสมบัติของ LLM Model ล่าสุดของบริษัทที่ถูกประกาศออกมาในช่วงเดือนกันยายนปี พ.ศ. 2568 โดยประกอบไปด้วย 4 โมเดลนั่นก็คือ kgpt-multimodal โมเดลที่นอกจากการรับเป็นตัวอักษรแล้วยังสามารถรับเข้าเป็นรูปภาพได้อีกด้วย โดยมีขนาด 32,000 tokens ใช้งานได้หลายงานหลายประเภทมากขึ้น และมี kgpt-mini-text และ kgpt-reasoning-text ที่สามารถรับได้เฉพาะข้อมูลแบบตัวอักษร โดย kgpt-reasoning-text สามารถรับ token ได้สูงสุดถึง 64,000 tokens เหมาะสำหรับรับข้อความขนาดใหญ่ และซับซ้อนกว่า kgpt-mini-text และโมเดลตัวสุดท้ายที่มีใช้สำหรับการ Embedding ข้อความในกระบวนการ RAG นั่นเอง

2.2.15 การนำเสนอเชิงเทคนิคและการถ่ายทอดความรู้

ได้ฝึกสังเคราะห์ข้อมูลเชิงเทคนิคที่ซับซ้อนและนำเสนอให้เข้าใจง่ายกับผู้ฟัง การจัดทำสื่อสำหรับนำเสนอ

2.2.16 การจัดการโครงการและการวางแผนงาน

พัฒนาทักษะในการสร้างและจัดทำเอกสารโครงสร้างงาน การวางแผนงานย่อย และการรายงานความคืบหน้าแก่ Mentor อย่างสม่ำเสมอ เพื่อให้โครงการมีทิศทางที่ชัดเจนและโปร่งใส

2.2.17 การเป็นส่วนหนึ่งขององค์กร (Corporate Citizenship)

จากการเข้าร่วมกิจกรรมทั้งด้านวิชาชีพและสังคมของบริษัท ทำให้ได้เรียนรู้ และได้ประสบการณ์ผ่านการทำงานในองค์กร ได้แก่

- การเข้าร่วมปฐมนิเทศพนักงานใหม่
- การทำ Training ด้าน HR เช่น ระบบลงเวลางาน
- การร่วมกิจกรรม CSR ปลุกป่าชายเลน
- การเรียนรู้ผ่านคอร์สภายในองค์กรอย่าง Symphony

3. วิธีดำเนินการ

3.1 แผนการดำเนินงาน

โครงการนี้ถูกดำเนินการอย่างเป็นระบบตลอดระยะเวลา 4 เดือน โดยเริ่มจากการวิจัยและการตั้งค่าพื้นฐาน ไปจนถึงการพัฒนาพีเจอร์ขั้นสูงและการปรับปรุงขั้นสุดท้าย แผนงานตามลำดับเวลานี้แสดงถึงงานสำคัญและเหตุการณ์สำคัญที่เกิดขึ้นในแต่ละเดือน เพื่อแสดงให้เห็นถึงพัฒนาการของโครงการ

แผนปฏิบัติงานสหกิจศึกษา เดือนที่ 1

ช่วงระยะเวลา: 18 สิงหาคม พ.ศ.2568 – 8 กันยายน พ.ศ.2568

ตารางที่ 4 แสดงถึงงานในแต่ละวันในระยะเวลา 4 สัปดาห์ ในเดือนที่ 1

ลำดับ	หัวข้องาน	สัปดาห์ 1					สัปดาห์ 2					สัปดาห์ 3					สัปดาห์ 4				
1.	Orientation, ขอสสิทธิการเข้าระบบต่างๆ, ติดตั้งโปรแกรม																				
2.	ศึกษา Tools และ API ที่มีอยู่ในบริษัท																				
3.	ศึกษา RAG Concept และวิธี Implement LLM																				
4.	จัดทำ Knowledge Sharing เกี่ยวกับ API ของบริษัท																				
5.	ทดลองดึง API โดยใช้ Python																				
4.	พัฒนาระบบจัดการไฟล์ Simulink และสร้าง Requirements (Python Prototype)																				
5.	เข้าฟังประชุม หรือ รายงานความคืบหน้าของงานที่ได้รับมอบหมาย																				
6.	กิจกรรมร่วมกับทางบริษัท																				
7.	อัปเดตโปรเจกต์บน Gitlab																				

จากตารางที่ 4 แสดงถึงการทำงานในเดือนแรกได้เน้นไปที่การเริ่มงานจากการเรียนภาพรวมของรู้บริษัท การวิจัย รวมไปถึงเครื่องมือหรือเทคโนโลยีและวิธีการสร้างพื้นฐานทางเทคนิคของโครงการ เป้าหมายหลักคือการ

ทำความเข้าใจกับระบบที่มีอยู่ของบริษัท และสร้างต้นแบบการทำงานสำหรับ workflow ที่ใช้ RAG (Retrieval-Augmented Generation) ได้แก่

- เข้าร่วมการปฐมนิเทศบริษัท การฝึกอบรม HR และตั้งค่าระบบ ศึกษา API ของ File Hub ภายในบริษัทและเครื่องมือ AI
- วิจัยแนวคิด RAG และวางแผน workflow ของโครงการ
- พบปัญหาเบื้องต้นที่การทำงานของ RAG ไม่สามารถประมวลผลไฟล์ทั้งหมดได้ จึงปรับกลยุทธ์ไปที่การแยกเนื้อหา XML ก่อน
- พัฒนา logic สำหรับการอัปโหลดไฟล์ และสร้าง flow แรกสำหรับการประมวลผลไฟล์ .slx ด้วย LLM ด้วย Python
- สร้างโครงสร้างข้อมูลแบบ Tree ใน backend เพื่อจัดการความสัมพันธ์ของไฟล์ XML จาก .slx

แผนปฏิบัติงานสหกิจศึกษา เดือนที่ 2

ช่วงระยะเวลา: 15 กันยายน พ.ศ.2568 – 10 ตุลาคม พ.ศ.2568

ตารางที่ 5 แสดงถึงงานในแต่ละวันในระยะเวลา 4 สัปดาห์ ในเดือนที่ 2

ลำดับ	หัวข้องาน	สัปดาห์ 1				สัปดาห์ 2				สัปดาห์ 3				สัปดาห์ 4			
1.	ศึกษา framework สำหรับระบบ																
2.	พัฒนาระบบจัดการไฟล์ Simulink และสร้าง Requirements																
3.	เข้าฟังประชุม หรือ รายงานความคืบหน้าของงานที่ได้รับมอบหมาย																
4.	อัปเดตโปรเจกต์บน Gitlab																
5.	เข้าร่วมฟัง Knowledge Sharing																

จากตารางที่ 5 ในเดือนที่ 2 นั้นได้เน้นไปที่การพัฒนาฐานข้อมูลและ UI หลัก เมื่อวางรากฐานเสร็จ เดือนที่สองมุ่งเน้นไปที่การสร้างเว็บไซต์สำหรับจัดเก็บไฟล์โดยใช้เฟรมเวิร์ก Svelte, Express, TypeScript เป็นหลัก โดยสามารถอัปโหลดไฟล์ .SLX ผ่าน UI บนเว็บไซต์ และเรียก Python Script ผ่านการใช้โมดูลมาตรฐานของ Node อย่าง child process และพัฒนา UI หลักของแอปพลิเคชัน โดยเน้นการจัดการไฟล์และการแชร์เนื้อหาที่สร้างขึ้น

- ออกแบบฐานข้อมูล MySQL ใหม่เพื่อรองรับฟีเจอร์ของแอปพลิเคชัน

- พัฒนาโครงสร้าง UI หลัก
- สร้างระบบจัดการไฟล์และโพลเดอร์บนเว็บไซต์แบบครบวงจร (CRUD: Create, Read, Update, Delete)

แผนปฏิบัติงานสหกิจศึกษา เดือนที่ 3

ช่วงระยะเวลา: 13 ตุลาคม พ.ศ.2568 – 07 พฤศจิกายน พ.ศ.2568

ตารางที่ 6 แสดงถึงงานในแต่ละวันในระยะเวลา 4 สัปดาห์ ในเดือนที่ 3

ลำดับ	หัวข้องาน	สัปดาห์ 1					สัปดาห์ 2					สัปดาห์ 3					สัปดาห์ 4				
1.	ศึกษาเพิ่มเติม																				
2.	พัฒนาระบบจัดการไฟล์ Simulink และสร้าง Requirements																				
3.	พัฒนาระบบแปลง Markdown Language เป็น Word Format																				
3.	เข้าฟังประชุม หรือ รายงานความคืบหน้าของงานที่ได้รับมอบหมาย																				
4.	อัปเดตโปรเจกต์บน Gitlab																				
5.	เข้าร่วมฟัง Knowledge Sharing																				
6.	จัดทำ Knowledge Sharing เกี่ยวกับ KGPT API และ File Hub API																				

จากตารางที่ 6 ในเดือนที่ 3 มีการสร้างฟังก์ชันใหม่อย่างการสร้าง File Word / PDF และมีการประเมินผลลัพธ์จาก LLM และเริ่มมีการศึกษาโครงสร้างของโปรเจกต์เว็บไซต์ Hil Commissioning ที่กำลังจะเข้าไปรวมด้วย

- พัฒนาฟีเจอร์ “Save Requirement” และ “Share Requirement” พร้อมตัวเลือกการส่งออก Word และ PDF รุ่นแรก
- แก้ไขข้อผิดพลาดจำนวนมากในระบบไฟล์และโพลเดอร์เพื่อให้มั่นใจในความเสถียรและการจัดการข้อมูลที่ถูกต้อง
- ปรับปรุงฟีเจอร์ส่งออก Word และ PDF อย่างมาก โดยเพิ่มการจัดรูปแบบละเอียด เช่น ตาราง หัวข้อ ฟอนต์ และสไตล์ข้อความ ตามต้นแบบไฟล์ที่ได้มา

- พัฒนาด้านแบบระบบประเมินคุณภาพของ Requirement ที่สร้างขึ้น

แผนปฏิบัติงานสหกิจศึกษา เดือนที่ 4

ช่วงระยะเวลา: 10 พฤศจิกายน พ.ศ.2568 – 8 ธันวาคม พ.ศ.2568

ตารางที่ 7 แสดงถึงงานในแต่ละวันในระยะเวลา 4 สัปดาห์ ในเดือนที่ 4

ลำดับ	หัวข้องาน	สัปดาห์ 1					สัปดาห์ 2					สัปดาห์ 3					สัปดาห์ 4				
1.	ศึกษาเพิ่มเติม																				
2.	พัฒนาระบบจัดการไฟล์ Simulink และสร้าง Requirements																				
3.	พัฒนาระบบ Chatbot เชื่อมกับ Hil Commissioning Project																				
4.	พัฒนาระบบ Tool Management และ Chatbot Setting																				
5.	เข้าฟังประชุม หรือ รายงานความคืบหน้าของงานที่ได้รับมอบหมาย																				
6.	อัปเดตโปรเจกต์ Gitlab																				
7.	เข้าร่วมฟัง Knowledge Sharing																				
8.	จัดทำรายงาน																				

จากตารางที่ 7 ในเดือนที่ 4 เป็นเดือนที่มีการเปลี่ยนไปสู่การพัฒนาโมเดลการโต้ตอบหลักของผู้ใช้หรือที่เรียกว่า Chatbot ซึ่งมีการเคลื่อนย้ายโปรเจกต์รวมกับโปรเจกต์เว็บไซต์ Hil Commissioning โดยมี Chatbot ว่าเป็นตัวเชื่อมสำหรับไฟล์ที่มีในเว็บไซต์ให้สามารถเอามาใช้ร่วมกับ Tool ที่มีได้ โดยได้มีการสร้าง Tool Setting เพื่อตอบสนองความต้องการนี้ และได้มีการจัดทำรายงานสำหรับส่งวิชาสหกิจศึกษาของทางมหาลัย โดยจะมีการจัดทำ LLM Setting ต่อไปสร้าง UI หลักของ Chatbot พร้อมระบบจัดการ Session และประวัติการสนทนา

- ปรับปรุงฐานข้อมูลอย่างต่อเนื่อง โดยเพิ่มตารางสำหรับ Session, ประวัติแชท, Tool และ Keyword เพื่อรองรับฟีเจอร์ใหม่
- ปรับปรุง UI/UX ของ Chatbot เช่น เพิ่มการอัปโหลดไฟล์แบบ Drag-and-Drop, กล่องข้อความขยายอัตโนมัติ และ Tooltip
- สร้าง Tool Management System ให้ผู้ใช้สามารถสร้าง แก้ไข ลบ และเปิดใช้งานเครื่องมือจาก UI

- ย้ายฟังก์ชันการแตกไฟล์จาก Python ไปเป็น TypeScript ใน Backend ของ Node.js เพื่อลดการพึ่งพาและรวมโค้ดให้เป็นหนึ่งเดียว
- ปรับสถาปัตยกรรมการสร้างไฟล์ Word จากการทำงานฝั่ง Client ไปเป็น API ฝั่ง Server ที่สร้างไฟล์และอัปโหลดไปยัง File Server
- พัฒนา logic สำหรับตรวจจับและใช้เครื่องมือโดยอิงจาก Keyword ที่พบใน Prompt ของผู้ใช้
- เดือนสุดท้ายมุ่งเน้นไปที่การปรับปรุงประสบการณ์ผู้ใช้ การปรับสถาปัตยกรรม Backend และการทำให้ฟีเจอร์หลักสมบูรณ์
- จัดทำเล่มรายงานส่งให้ทางมหาลัย และเอกสารเกี่ยวกับระบบเพื่อส่งมอบงานให้แก่ทางบริษัทดำเนินการพัฒนาต่อไป

4. ผลปฏิบัติงาน

4.1 ผลงานโครงการหลัก

งานโครงการนี้สิ้นสุดลงด้วยการสร้างเว็บแอปพลิเคชันที่ทำงานได้จริงที่สามารถรองรับเครื่องมือเพื่อการต่อยอดต่อไปในอนาคต เพื่อช่วยปรับปรุงกระบวนการทำงานด้านวิศวกรรมสำหรับการสร้างเอกสารข้อกำหนดแอปพลิเคชันนี้ผสานรวมระหว่างระบบแสดงผลผ่านเว็บไซต์ร่วมกับระบบที่นำ Large Language Model (LLM) มาประยุกต์ใช้ผ่านการเรียก API ที่บริษัทได้มีการพัฒนาไว้แล้ว โดยประกอบด้วยฟีเจอร์ที่สร้างมาเพื่อตอบโจทย์วัตถุประสงค์หลักของโครงการ

4.1.1 ชุดเทคโนโลยีและเครื่องมือที่ถูกนำมาใช้สำหรับพัฒนาระบบ

ตารางที่ 8 แสดงถึงชุดเทคโนโลยีและเครื่องมือที่ถูกนำมาใช้สำหรับพัฒนาระบบ

หมวดหมู่	เครื่องมือ
ภาษาโปรแกรม	TypeScript, JavaScript, Python, SQL
Backend Frameworks	Node.js, Express, Fast API
Frontend Frameworks	Svelte
ฐานข้อมูล	MySQL, MySQL Workbench
กระบวนการพัฒนา	GitLab, Virtual Environments, MVC Architecture, Bun, SoapUI, draw.io
ระบบภายในองค์กร	Company AI API (KGPT), API FILE HUB,

จากตารางที่ 8 แสดงถึงเครื่องมือที่ได้เรียนรู้เพิ่มเติมผ่านการพัฒนาและสร้างระบบบนเว็บไซต์ในครั้งนี้ โดยทำการจัดเครื่องมือให้แยกออกมาตามหมวดหมู่ดังนี้

ภาษาโปรแกรม

- TypeScript และ JavaScript: ใช้สำหรับพัฒนาส่วนหลังบ้าน (Backend) ของเว็บไซต์ โดย TypeScript ช่วยเพิ่มความปลอดภัยของโค้ดด้วยการตรวจสอบชนิดข้อมูล (Type Safety) ส่วน JavaScript เป็นภาษาหลักที่ทำงานร่วมกับ Node.js
- Python: ใช้สำหรับงานที่เกี่ยวข้องกับ AI และ LLM (Large Language Models) รวมถึงการสร้าง API ที่ต้องการประสิทธิภาพสูง
- SQL: ใช้สำหรับการจัดการฐานข้อมูล เช่น การสร้าง แก้ไข ดึงข้อมูล และลบข้อมูลจาก Database

Backend Frameworks

- Node.js: รัน JavaScript บนฝั่งเซิร์ฟเวอร์ ทำให้สามารถสร้างแอปพลิเคชันที่มีประสิทธิภาพสูง และรองรับการทำงานแบบ asynchronous
- Express: Framework ยอดนิยมบน Node.js สำหรับสร้าง RESTful API และจัดการ routing อย่างง่าย
- FastAPI: Framework บน Python ที่เน้นความเร็วและรองรับการสร้าง API ที่มีประสิทธิภาพสูง พร้อมระบบตรวจสอบข้อมูลอัตโนมัติ โดยภายหลังได้เปลี่ยนไปใช้ Express เพื่อความง่ายต่อการพัฒนา และ ความเข้ากันของระบบ

Frontend Frameworks

- Svelte: Framework สำหรับสร้าง UI ที่มีประสิทธิภาพสูง โดยคอมไพล์โค้ดไปเป็น JavaScript ที่เบาและเร็ว เหมาะสำหรับเว็บแอปพลิเคชันที่ต้องการความเร็วและประสิทธิภาพใช้งานที่ดี

ฐานข้อมูล

- MySQL: ระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ที่นิยมใช้สำหรับเก็บข้อมูลของแอปพลิเคชัน
- MySQL Workbench: เครื่องมือสำหรับออกแบบและจัดการฐานข้อมูล MySQL ผ่าน GUI ทำให้การทำงานสะดวกขึ้น

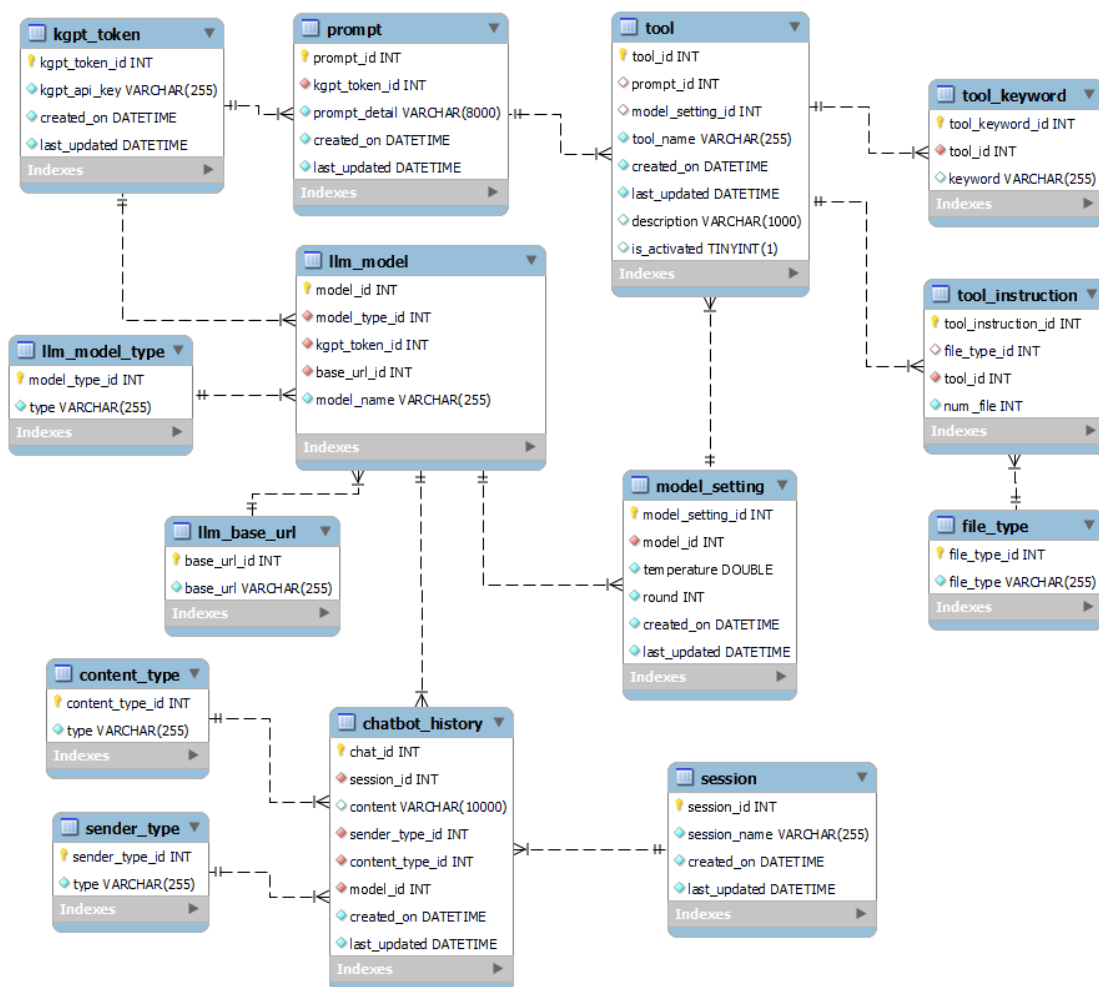
กระบวนการพัฒนา

- GitLab: ใช้สำหรับการจัดการเวอร์ชันของโค้ด (Version Control) และ CI/CD Pipeline
- Virtual Environments: ใช้สำหรับแยกสภาพแวดล้อมของ Python เพื่อป้องกันการชนกันของ dependency
- MVC Architecture: รูปแบบการออกแบบซอฟต์แวร์ที่แยกส่วน Model, View, Controller เพื่อความเป็นระเบียบและง่ายต่อการดูแล
- Bun: Runtime ที่ทำงานคล้าย Node.js แต่มีความเร็วสูงกว่า เหมาะสำหรับงานที่ต้องการประสิทธิภาพ
- SoapUI: เครื่องมือสำหรับทดสอบ API โดยเฉพาะ SOAP และ REST
- draw.io: ใช้สำหรับสร้าง Diagram เช่น Flowchart, ER Diagram เพื่อออกแบบระบบ

ระบบภายในองค์กร

- Company AI API (KGPT): API สำหรับเรียกใช้งาน AI ภายในองค์กร เช่น การประมวลผลข้อความ
- API FILE HUB: ระบบสำหรับจัดการไฟล์และ API ภายในองค์กร เพื่อให้การแลกเปลี่ยนข้อมูลเป็นไปอย่างปลอดภัยและมีประสิทธิภาพ

4.1.2 ฐานข้อมูล



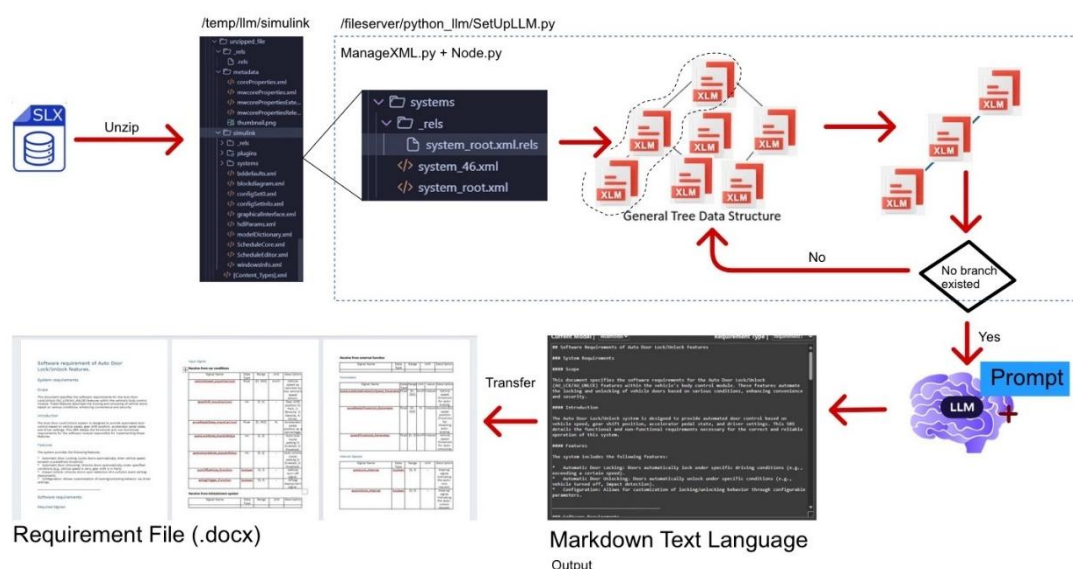
ภาพที่ 7 Entity-Relationship Diagrams

จากภาพที่ 7 แสดงถึงแผนภาพแสดงความสัมพันธ์ของแต่ละตารางใน Schema LLM โดยได้ออกแบบและพัฒนาระบบฐานข้อมูลโดยใช้ MySQL เพื่อจัดเก็บและบริหารข้อมูลของระบบ Chatbot และ Tool Management การตั้งค่าของส่วนต่างๆเพื่อให้ User สามารถตั้งค่าได้ผ่าน User Interface ที่กำลังออกแบบและพัฒนาอยู่ พร้อมทั้งมีการกำหนดข้อจำกัด (Constraints) เพื่อรักษาความถูกต้องและครบถ้วนของข้อมูล

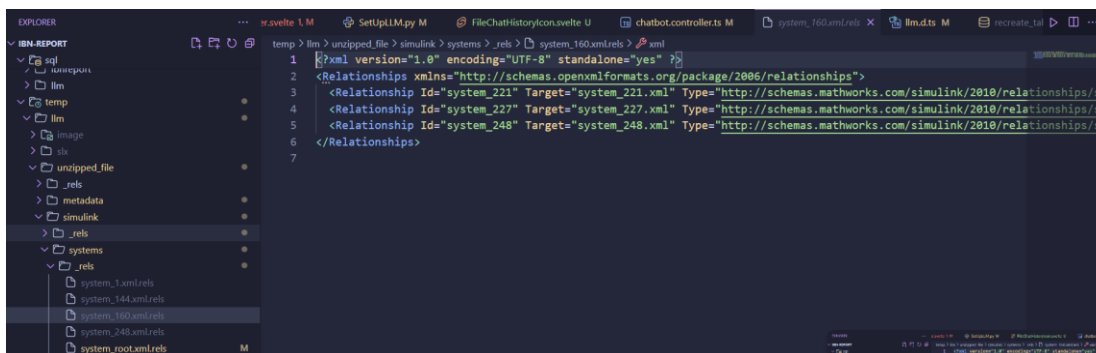
4.1.3 คำอธิบายเพิ่มเติมของระบบที่ถูกพัฒนาขึ้นในโครงการหลัก

4.1.3.1 ระบบการสร้าง Requirement จาก AI

ระบบนี้ถูกพัฒนามาเพิ่มเติมจาก KGPT ที่มีอยู่เดิมไม่สามารถรับและประเมินผล ไฟล์ .slx จากลูกค้าได้จึงจำเป็นต้องนำ KGPT API มาพัฒนาต่อยอดต่อ สำหรับฟังก์ชันหลักของระบบคือความสามารถในการประมวลผลไฟล์ .slx (Simulink Model) ซึ่งเป็นไฟล์ .zip ประเภทหนึ่งของ MATLAB ที่มีการเก็บข้อมูลของไฟล์ .xml อย่างเป็นระบบเพื่อนำกลับไปแสดงเป็นแผนภาพอีกครั้งในแอปพลิเคชัน MATLAB จึงทำให้ไฟล์ .slx เป็นไฟล์ที่มีความซับซ้อน การดึงข้อมูลสำคัญจากโครงสร้าง XML ภายใน และใช้โมเดล LLM ที่เป็นกรรมสิทธิ์ของบริษัทร่วมกับแนวทาง Retrieval-Augmented Generation (RAG) สิ่งนี้ช่วยแก้ปัญหากระบวนการสร้างเอกสารข้อกำหนดแบบแมนนวลที่ใช้เวลานานและมีโอกาสผิดพลาด โดยทำให้สามารถสร้างเอกสารที่มีโครงสร้างและสอดคล้องกับบริบทได้โดยอัตโนมัติจากโมเดลวิศวกรรม

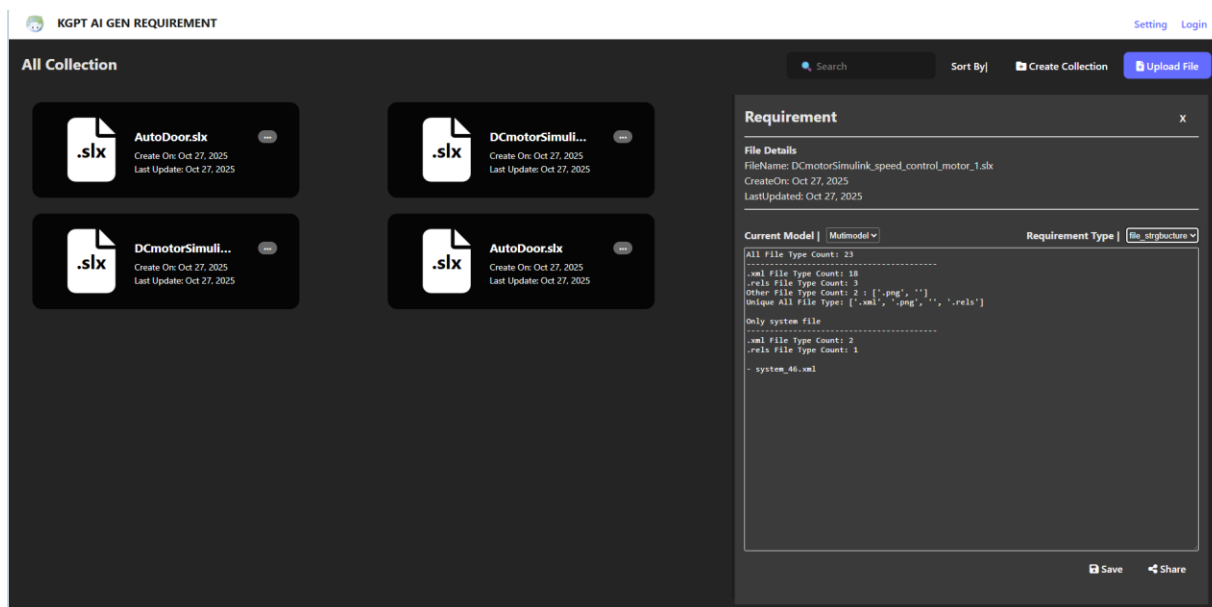


ภาพที่ 8 ภาพระบบเบื้องต้นของ การสร้าง Software Requirement จาก AI



ภาพที่ 9 ตัวอย่างไฟล์แสดงความสัมพันธ์ของระบบ

จากภาพที่ 8 แสดงถึงกระบวนการการทำงานที่เริ่มต้นด้วยการรับไฟล์ .slx ที่รับเข้ามาจาก user ผ่านทางหน้า User Interface ของระบบเว็บไซต์ HIL Commissioning หลังจากนั้นจะมีการเรียก API postChat จากไฟล์ controller จาก Backend ในโพลเดอร์ restapi เพื่อเก็บค่าประวัติการสนทนาลง Database และมีการ เชื่อมไปที่ ฟังก์ชัน Tool Selection เพื่อเลือก Tool ที่เหมาะสมขึ้นมาในที่นี้ก็คือ Simulink Requirement โดยจะมีการแตกไฟล์ออกเป็นโครงสร้างดังรูปลงใน temp/llm/unzipfile และเรียก API ที่อยู่ใน file server เพื่อเรียก code python SetUpLLM.py ขึ้นมาเพื่อบันทึกค่าความสัมพันธ์ของไฟล์ .slx ที่แตกออกมา โดยจะเลือกเก็บเฉพาะข้อมูลใน systems folder ที่มีข้อมูลที่เกี่ยวข้องในสำหรับการนำมาสร้าง Software Requirement โดยจะเริ่มอ่านความสัมพันธ์จากไฟล์ system_root.xml.rels ใน folder _rels ตามภาพที่ 9 ซึ่งจะมีชื่อไฟล์ที่เป็น node ลูกกระบวย โดยจะได้เก็บข้อมูลในรูปแบบ Tree ไปเรื่อยๆจนครบ และนำข้อมูลแต่ละกิ่งมารวมเข้ากับ Prompt Template ที่เตรียมไว้และป้อนข้อมูลโดยการส่ง KGPT API ไปที่ LLM model ที่ระบุไว้ และวนจนครบทุกกิ่งของโครงสร้างข้อมูลโดยให้ LLM ค่อยๆเติมข้อมูลในส่วนที่เพิ่มไปแต่ละครั้งเป็นกระบวนการที่ทำมาเพื่อไม่ให้เกิดปัญหาการเกินออกมาของ Token และเมื่อได้ผลลัพธ์ออกมาแล้วจะนำไปเข้าสู่ระบบประมวลผล และระบบแปลงเป็นไฟล์ .docx บันทึกเก็บไว้ภายใต้ไฟล์ .slx ต่อไป



ภาพที่10 ภาพต้นแบบระบบสร้าง Requirement V.1

จากภาพที่ 10 ในช่วงแรกของการพัฒนานั้นระบบการสร้าง Requirement เป็นระบบที่ถูกสร้างแยกออกจาก HIL Commissioning ที่หลังจากอัปโหลดไฟล์ Requirement นั้นจะส่งค่าเป็น

Markdown Language กลับมาโดยผู้ใช้งานสามารถแก้ไขผลลัพธ์ได้ผ่านหน้าเว็บไซต์ และสามารถแชร์เพื่อส่งออกข้อมูลเป็นไฟล์ word และ pdf ภายหลัง

Software requirement of Auto Door Lock/Unlock features.

System requirements

Scope

This document specifies the software requirements for the Auto Door Lock/Unlock (AU LOCK/AU_UNLOCK) features within the vehicle's body control module. These features automate the locking and unlocking of vehicle doors based on various conditions, enhancing convenience and security.

Introduction

The Auto Door Lock/Unlock system is designed to provide automated door control based on vehicle speed, gear shift position, accelerator pedal state, and driver settings. This SRS details the functional and non-functional requirements for the software module responsible for implementing these features.

Features

The system provides the following features:

- * Automatic Door Locking: Locks doors automatically when vehicle speed exceeds a predefined threshold.
- * Automatic Door Unlocking: Unlocks doors automatically under specified conditions (e.g., vehicle speed is zero, gear shift is in Park).
- * Impact Unlock: Unlocks doors upon detection of a collision event (airbag deployment).
- * Configuration: Allows customization of locking/unlocking behavior via driver settings.

Software requirements

Required Signals

Input Signal

Receive from car conditions

Signal Name	Data Type	Range	Unit	Description
ความเร็วของรถ (km/h)	Float	[0, 200]	km/h	Vehicle speed as reported by the vehicle's speed sensor.
ตำแหน่งเกียร์ (Park, 2, Reverse, 3, Neutral, 4, Drive)	Int	[1, 4]	-	Gear shift position (1: Park, 2: Reverse, 3: Neutral, 4: Drive).
ตำแหน่งคันเร่ง (0-100%)	Float	[0, 100]	%	Accelerator pedal position as a percentage.
โหมดการขับขี่ (Enabled, 2; Disabled)	Int	[1, 2]	-	Auto lock mode setting (1: Enabled, 2: Disabled).
โหมดการปลดล็อค (Enabled, 2; Disabled)	Int	[1, 2]	-	Auto unlock mode setting (1: Enabled, 2: Disabled).
สัญญาณไฟฉุกเฉิน (0, 1)	Boolean	[0, 1]	-	Vehicle turn-off signal.
สัญญาณถุงลมนิรภัย (0, 1)	Boolean	[0, 1]	-	Airbag deployment signal.

Receive from infotainment system

Signal Name	Data Type	Range	Unit	Description

Receive from external function

Signal Name	Data Type	Range	Unit	Description

Parameters

Signal Name	Data Type	Range	Unit	Value	Description
ความเร็วของรถ (km/h)	Float	[0, 200]	km/h	<value>	Vehicle speed threshold for auto-locking.
ตำแหน่งคันเร่ง (0-100%)	Float	[0, 100]	%	<value>	Accelerator pedal position threshold for disabling auto-locking.
โหมดการขับขี่ (Enabled, 2; Disabled)	Int	[0, 1]	km/h	<value>	Vehicle speed threshold for auto-unlocking.

Internal Signals

Signal Name	Data Type	Range	Unit	Description
สัญญาณล็อคประตู (0, 1)	Boolean	[0, 1]	-	Internal signal indicating the auto-lock request.
สัญญาณปลดล็อคประตู (0, 1)	Boolean	[0, 1]	-	Internal signal indicating the auto-unlock request.

ความเร็วของรถ (km/h)	Float	[0, 200]	km/h	Speed threshold for auto-unlock.
stop speed threshold	Float	[0, 200]	h	Stop speed threshold

Internal Signals

Signal Name	Data Type	Range	Unit	Description
สัญญาณล็อคประตู (0, 1)	Boolean	[0, 1]	-	Internal signal for auto lock function.
สัญญาณปลดล็อคประตู (0, 1)	Boolean	[0, 1]	-	Internal signal for auto unlock function.
สัญญาณไฟฉุกเฉิน (0, 1)	Boolean	[0, 1]	-	Internal signal for impact unlock function.

Output Signals

Signal Name	Data Type	Range	Unit	Description
door_lock_enable_output	Boolean	[0, 1]	-	Output signal to lock the doors.
door_unlock_enable_output	Boolean	[0, 1]	-	Output signal to unlock the doors.

Requirement

ID	Type	Description	Test Method
SW_REQ_1	Functional	Function Auto Lock Enable on speed	Dynamic Testing
SW_REQ_2	Functional	Function Auto Lock Enable on speed	Dynamic Testing

SW_REQ_1	Requirement	the vehicle doors when the vehicle speed exceeds a predefined threshold. code logic: IF speed > speed_threshold == 1 AND speed_sensor_value > speed_threshold THEN auto_lock_enable = TRUE; ELSE auto_lock_enable = FALSE; ENDIF	Dynamic Testing (Unit test)
SW_REQ_2	Requirement	Function Auto Lock Enable on Park # This function enables the automatic locking of the vehicle doors when the gear is shifted to drive. code logic: IF auto_lock_mode_setting == 1 AND gear_shift_position == 4 AND speed_sensor_value > speed_threshold THEN auto_lock_enable = TRUE; ELSE auto_lock_enable = FALSE; ENDIF	Dynamic Testing (Unit test)
SW_REQ_3	Requirement	Function Auto Unlock Enable on Park # This function enables the automatic unlocking of the vehicle doors when the gear is shifted to Park. code logic: IF auto_unlock_mode_setting == 1 AND gear_shift_position == 1 AND speed_sensor_value > speed_threshold THEN auto_unlock_enable = TRUE; ELSE auto_unlock_enable = FALSE; ENDIF	Dynamic Testing (Unit test)
SW_REQ_4	Requirement	Function Auto Unlock Enable on Vehicle Off # This function enables the automatic unlocking of the vehicle doors when the vehicle is turned off. code logic: IF auto_unlock_mode_setting == 1 AND vehicle_turn_off_signal == TRUE THEN auto_unlock_enable = TRUE; ENDIF	Dynamic Testing (Unit test)

SW_REQ_5	Requirement	Function Impact Unlock # This function enables the automatic unlocking of the vehicle doors when the airbag is triggered. code logic: IF airbag_triggered == TRUE THEN auto_unlock_enable = TRUE; ELSE auto_unlock_enable = FALSE; ENDIF	Dynamic Testing (Unit test)
SW_REQ_6	Requirement	Function Door Lock # This function locks the vehicle doors based on the auto_lock_enable signal. code logic: IF auto_lock_enable == TRUE THEN door_lock_enable = TRUE; ELSE door_lock_enable = FALSE; ENDIF	Dynamic Testing (Functional test)
SW_REQ_7	Requirement	Function Door Unlock # This function unlocks the vehicle doors based on the auto_unlock_enable signal. code logic: IF auto_unlock_enable == TRUE THEN door_unlock_enable = TRUE; ELSE door_unlock_enable = FALSE; ENDIF	Dynamic Testing (Functional test)

ภาพที่ 11 ภาพตัวอย่างผลลัพธ์สุดท้าย

จากภาพที่ 11 เป็นภาพประกอบของผลลัพธ์สุดท้ายก็คือไฟล์ Microsoft Word ของไฟล์

Software Requirements ที่ประกอบไปด้วย System Requirements และ Software Requirements ซึ่ง System Requirements จะประกอบไปด้วย Scope, Introduction, และ Features ส่วน Software Requirements จะประกอบไปด้วยส่วนของ Required Signals ที่จะมีส่วน Input Signal (แบ่งเป็น ตาราง Receive from car condition, Receive from infotainment system และ Receive from external function), ตาราง Internal Signals, ตาราง Output Signals, ตาราง Parameter และ ตาราง Requirement ที่นำไปใช้สำหรับการประกอบการเขียน test แต่ละระบบ

```

class SetUpLLM:

    # สร้าง LLM
    llm = ChatOpenAI(
        model = "kgpt-multimodal",
        base_url = "https://llm-proxy.kpit.com",
        api_key = os.getenv("OPENAI_API_KEY"),
        temperature = 0.7,
        seed = 42
    )

    def __init__(self, task_name: str, unzipped_path: str):
        # สร้าง Prompt Template
        self.prompt = ChatPromptTemplate.from_messages(
            [
                (
                    "system",
                    "You are an {expertise} who writes requirements from Simulink models. "
                    "'You will analyze .xlm files that i have uploaded for you using RAG and answer the user's question. "
                    "I need your assistance in drafting a Software Requirements Specification (SRS) document. "
                    "I will provide details incrementally, and I need you to refine them and format them according to the structure outlined below. "
                    "The goal is a formal SRS - please minimize conversational language and focus on technical specification.'",
                ),
                (
                    "human",
                    "{context}\n\n"
                    "Question: {question}",
                ),
            ]
        )

```

ภาพที่ 12 ภาพของ class SetUpLLM

จากภาพที่ 12 เป็น Object ของ SetUpLLM ที่เขียนมาสำหรับการรองรับการขยายการใช้งานเพิ่มเติมในอนาคต โดยแบ่งได้เป็น 2 ส่วนคือ ส่วนที่กำหนดค่าตัวแปร llm และ ส่วน prompt template ในส่วนของ llm

มีการใช้ Chat OpenAI จาก Lang Chain เพื่อเชื่อมต่อกับโมเดล LLM model: "kgpt-multimodal" ซึ่งเป็นโมเดลภายในองค์กร รองรับข้อความและมัลติมีเดีย โดยมีการปรับค่าไว้เบื้องต้น ดังนี้

- ตัวแปร base_url URL สำหรับ proxy เรียกใช้งาน LLM
- ตัวแปร api_key เพื่อเป็นตัวยืนยันสิทธิการเข้าถึงข้อมูล โดยดึงจาก environment variable เพื่อความปลอดภัย
- ตัวแปร temperature: 0.7 ที่เป็ค่าสำหรับควบคุมความสร้างสรรค์ของคำตอบ
- ตัวแปร seed สำหรับ 42 ทำให้ผลลัพธ์ reproducible
- ตัวแปร ใช้ ChatPromptTemplate เพื่อสร้าง prompt แบบ structured conversation

ในส่วนของ Prompt Template

ได้แบ่งออกเป็นสองส่วนคือ system message และ human message

- ส่วนของ system message

มีการกำหนด Prompt Template ให้กำหนดบทบาทของโมเดลผ่านตัวแปร {expertise} และใส่ในส่วนของการอธิบายงานด้วยข้อความว่า วิเคราะห์ไฟล์ XML โดยใช้ RAG และตอบคำถาม เน้นความเป็นทางการ (formal) และลดภาษาสนทนา

- ส่วนของ human message

ตัวแปร {context} ข้อมูลจากไฟล์ XML หรือเนื้อหาที่เกี่ยวข้อง และตัวแปร {question} คำถามที่ต้องการให้โมเดลตอบ ซึ่งสามารถดูส่วนเติมได้จากรูปที่ 13 เพิ่มเติมต่อไปในด้านล่าง

Data → Data Preparation → Lang Chain (Prompt + LLM) → Generate → Evaluate

ภาพที่ 14 สรุปภาพรวมของการทำงานของ method answer_by_prompt ใน Class
SetupLLM

จากภาพที่ 13 และ 14 จากโค้ดที่แนบมา สามารถสรุปเป็น 3 ขั้นตอนหลักของการทำงานกับ LLM ผ่าน Lang Chain ได้ดังนี้

1. การเตรียมข้อมูล (Data Preparation)

จุดประสงค์เพื่อให้ข้อมูลพร้อมสำหรับการส่งเข้าโมเดล โดยมีการแปลงเป็นข้อความที่มีบริบทครบถ้วนก่อนนำไปรวมกับ Prompt Template ที่เตรียมไว้ก่อนเข้าสู่ Lang chain เพื่อนำส่งให้ LLM ประมวลผลเป็นการถัดไป โดยจะมีขั้นตอนและกระบวนการตามลำดับนี้

- โหลดไฟล์ XML และสร้างโครงสร้างต้นไม้ (Tree Structure)
- ดึง Root Node ขึ้นมาสำหรับค้นหาค่า Leaf Nodes ทั้งหมด
- สร้างรายการเนื้อหาที่จัดรูปแบบ (format_requirement_content_lists) โดยรวมข้อมูลจากการ trace node ขึ้นไป
- เก็บผลโครงสร้างข้อมูลในรูปแบบโครงสร้างต้นไม้ไว้สำหรับตรวจสอบความถูกต้องได้ในอนาคต

2. ต่อเข้ากับ LangChain (LLM Integration)

- เริ่มจากการสร้าง Chain โดยใช้ self.prompt | SetUpLLM.llm | StrOutputParser()
- ส่งข้อมูลเข้าโมเดลผ่าน chain.invoke() พร้อม context และ question ที่มีรายละเอียดการจัดรูปแบบ (Format, Table Rules) เป็นการใช้ prompt ในรูปแบบ RACE
- ใช้ LLM เพื่อสร้าง Software Requirements Specification (SRS) ตามโครงสร้างที่กำหนด
- ทำงานแบบวนซ้ำเพื่อรวบรวมผลลัพธ์จากรอบก่อนหน้าเพื่อปรับปรุงเนื้อหาให้ครบถ้วน

3. ประเมินผล (Evaluation)

- ใช้ฟังก์ชัน evaluate_report() โดยมีการส่งค่า
 - Checklist ประกอบไปด้วยค่า Signal Name ของ parameter และ ค่า ID requirement ของ Requirement เข้าไปตรวจสอบ
 - Required Headings เป็นค่าคงที่ที่เก็บหัวข้อในเอกสารไว้ เพื่อนำไปส่งต่อเพื่อประเมินว่าหัวข้อสำคัญครบตาม Software Requirement หรือไม่
 - Sources ตรวจสอบความสอดคล้องกับข้อมูลต้นทาง
- แสดงสรุปคะแนนด้วย print_score_summary(metrics)

4.1.3.2 การพัฒนา Prompt ในรูปแบบ RACE

ตารางที่ 9 การพัฒนา Prompt ในรูปแบบ RACE

องค์ประกอบ	เก็บค่าในตัวแปร	ค่าในตัวแปร
Role	expertise	"KPIT's AI Assistant for employees (automotive engineer expert)"
Action	ChatPromptTemplate	"You will analyze. xlm files that i have uploaded for you using RAG and answer the user's question. I need your assistance in drafting a Software Requirements Specification (SRS) document. I will provide details incrementally, and I need you to refine them and format them according to the structure outlined below"
Context	context	"The Simulink model contains blocks for signal processing and control logic.\n" +"This is subsystem part of them. I place content about the system in text format below. It's .xml file:\n\n" + format_requirement_content_list + "\n\nPlease add and edit on this content:\n" + SoftwareRequirementsPart1
Expectation	ChatPromptTemplate /question	ใน ChatPromptTemplate The goal is a formal SRS – please minimize conversational language and focus on technical specification. ใน question จะสามารถอ้างอิงได้จากภาพที่ 5

จากตารางที่ 9 เป็นการอธิบายกระบวนการที่ใส่ค่าลงไปใน Prompt Template ที่หลังเพื่อรองรับการปรับเปลี่ยนของข้อมูลไฟล์ที่แตกต่างกันออกไป โดยมีการออกแบบโดยใช้หลักการ RACE Framework มาประกอบโดย**บทบาทหน้าที่**ที่กำหนดให้ LLM เป็นการจำลองว่าเป็นผู้ช่วยของพนักงานวิศวกรที่มีความเชี่ยวชาญสายยานยนต์ ในส่วนของการ**กระทำ**คือจะระบุเป็นคำสั่งว่าให้ช่วยสร้างเอกสาร Software Requirement โดยการวิเคราะห์ข้อมูลจากที่แนบมาและช่วยสร้างเอกสารตามรูปแบบที่กำหนด ในส่วนของ**บริบท**จะกำหนดเป็นข้อมูลจากไฟล์ .xml ที่จัดรูปแบบมาแต่ละกิ่งของโครงสร้างข้อมูล และแนบ ผลลัพธ์ที่สร้างขึ้นมาก่อนหน้าให้ช่วยเติมข้อมูลเพิ่มลงไป ส่วนสุดท้ายก็คือ**ความคาดหวัง**

จะเป็นส่วนที่ใส่เงื่อนไขการสร้าง format ขึ้นมา หัวข้อ ตารางแต่ละตารางประกอบไปด้วยอะไรบ้าง และ คำจำกัดความเพิ่มเติม กฎการสร้างเอกสาร เป็นต้น

4.1.3.3 ระบบประเมินผลลัพธ์ของเอกสาร Software Requirements ในไฟล์ Evaluate.py

ตารางที่ 10 อธิบายตัวแปรในไฟล์ Evaluate.py

ชื่อตัวแปร	ประเภท	คำอธิบาย
SENT_SPLIT_RE	regular expressions (Object)	ใช้สำหรับ แยกประโยค (Sentence Split) จะทำงานเมื่อเจอ จุด ., เครื่องหมาย !, ? ตามด้วยช่องว่าง และตัวอักษรใหญ่หรือเลข
WORD_RE	regular expressions (Object)	ใช้สำหรับ แยกคำ (Tokenize) รองรับตัวอักษรภาษาอังกฤษ, ตัวอักษรพิเศษ (accent), ภาษาไทย (ก-ฃ), และตัวเลข

ตารางที่ 11 อธิบายฟังก์ชันในไฟล์ Evaluate.py

ชื่อฟังก์ชัน	คำอธิบาย	ประเภท input	ประเภท output
split_sentences	ถ้า text ว่าง จะ คืนค่า [] มีการใช้ SENT_SPLIT_RE.split() เพื่อแยกประโยค ลบช่องว่างรอบ ๆ และคืนเฉพาะประโยคที่ไม่ว่าง โดนจะคืนค่าเป็นรายการประโยคที่แยกแล้ว	text: str	List[str]
tokenize	ใช้ WORD_RE.findall(text) เพื่อดึงคำทั้งหมด แปลงเป็นตัวพิมพ์เล็ก (lower()) และคืนรายการคำ (tokens)	text: str	List[str]
unique_ordered	ทำหน้าที่ ลบค่าที่ซ้ำออกจากลิสต์ แต่ยังคงลำดับเดิม โดยใช้ set() เพื่อตรวจสอบว่าค่าเคยเจอแล้วหรือไม่	List[str]	List[str]

shingles	คือเทคนิคในงาน Text Similarity ที่ใช้สร้างชุดของ n-grams (เรียกว่า shingles) จากข้อความ เพื่อใช้วัดความคล้ายกันระหว่างเอกสารหรือข้อความ โดยเน้นการรักษาลำดับของคำ แบ่งข้อความออกเป็น กลุ่มคำต่อเนื่อง (n-grams) ขนาด n เช่น 3, 5 แต่ละกลุ่มเรียกว่า shingle เปรียบเทียบชุด shingles ของข้อความสองชุดเพื่อหาความคล้ายกัน	tokens: List[str], n: int = 5	Counter object (จากโมดูล collections ซึ่งเป็น dictionary แบบพิเศษที่เก็บ tuple ของ n คำต่อเนื่อง (shingle) เป็น key และจำนวนครั้งที่ปรากฏเป็น value)
jaccard	คำนวณ Jaccard Similarity ระหว่างสองเซตตามสมการ $J(A, B) = \frac{ A \cup B }{ A \cap B }$ ยกตัวอย่างเช่น set1 = {"apple", "banana", "orange"} set2 = {"banana", "orange", "grape"} print(jaccard(set1, set2)) จะได้ Output เท่ากับ 0.5 # เพราะ intersection = {"banana", "orange"} (2) # union = {"apple", "banana", "orange", "grape"} (4) # 2/4 = 0.5	(a: set, b: set)	Float [0,1]

จากตารางที่ 10 และ 11 เป็นตัวแปรและ ฟังก์ชันที่ถูกนำมาใช้ในกระบวนการประเมินผลอีกที โดยมีการสรุปค่ารวมทั้งหมดในค่า overall ซึ่งคำนวณมาจากการวัดผลดังนี้

```

# ----- readability (FK grade, just estimate) -----
# regex (regular expressions)
VOWELS = set(list("aeiouyAEIOUY"))
TH_VOWELS = set(list("ะจุณเฬรึ"))

def count_syllables_en(word: str) -> int:
    w = word.lower()
    if not w:
        return 0
    count, prev_vowel = 0, False
    for ch in w:
        is_v = ch in VOWELS
        if is_v and not prev_vowel:
            count += 1
        prev_vowel = is_v
    if w.endswith("e") and count > 1:
        count -= 1
    return max(count, 1)

def estimate_syllables(text: str) -> int:
    words = WORD_RE.findall(text)
    if not words:
        return 0
    en_like = sum(count_syllables_en(w) for w in words)
    th_like = sum(1 for ch in text if ch in TH_VOWELS)
    return en_like + max(th_like // 3, 0)

def flesch_kincaid_grade(text: str) -> float:
    sentences = split_sentences(text)
    words = WORD_RE.findall(text)
    if not sentences or not words:
        return 0.0
    syllables = estimate_syllables(text)
    n_sent = max(len(sentences), 1)
    n_words = max(len(words), 1)
    return 0.39 * (n_words / n_sent) + 11.8 * (syllables / n_words) - 15.59

```

ภาพที่ 15

จากภาพที่ 15 เป็นฟังก์ชันไว้ใช้สำหรับการประเมิน Readability เพื่อใช้ประเมินความอ่านยากของภาษาค่าที่ได้จากสูตร Flesch–Kincaid Grade Level จะเป็นตัวเลขที่ประมาณระดับชั้นเรียน (Grade Level) ของผู้อ่านที่เหมาะสมกับข้อความนั้น โดยมีรายละเอียดดังนี้

ตารางที่ 12 ช่วงค่าที่ได้จากการประเมิน Readability จากสูตร Flesch–Kincaid Grade Level

ช่วงคะแนน	ความหมาย
[0 , 1]	ง่ายมาก (เหมาะสำหรับเด็กเล็ก)
[1 , 5]	ง่าย (ระดับประถมศึกษา)

[6 , 8]]	ปานกลาง (มัธยมต้น)
[9 , 12]	ค่อนข้างยาก (มัธยมปลาย)
[13 , 16]	ยาก (ระดับมหาวิทยาลัย)
> 16	ยากมาก (งานวิชาการ, บทความเชิงลึก)

จากตารางที่ 12 เป็นการแสดงช่วงค่าที่ได้ และระดับความยากในการอ่านโดยเปรียบเทียบจากระดับการศึกษา จากการใช้สูตร

$$\text{FK Grade} = 0.39 \times \frac{\text{จำนวนประโยค}}{\text{จำนวนคำ}} + 11.8 \times \frac{\text{จำนวนคำ}}{\text{จำนวนพยางค์}} - 15.59$$

หมายความว่า ยิ่งค่า สูง ข้อความซับซ้อนมาก, ต้องใช้ระดับการศึกษา สูง ในการอ่าน ส่วนถ้ายิ่งค่า ต่ำ ข้อความอ่านง่าย, เหมาะกับผู้อ่านทั่วไป

ส่วนมากใช้สำหรับการประเมิน วัดความยากของข้อความภาษาอังกฤษ เพื่อให้เหมาะกับกลุ่มผู้อ่าน ใช้ใน การเขียนเนื้อหา เช่น บทความ, เอกสาร, คู่มือ ให้ตรงกับระดับความเข้าใจของผู้อ่าน, การประเมินคุณภาพเนื้อหา สำหรับการเรียนการสอน, การตลาด, UX writing

```

# ----- structure & headings -----
# regex (regular expressions)
HEADING_RE = re.compile(
    r"^\s{0,3}(\#\s+|[\0-9]+\.\s+|[-*]\s+)?([A-Za-z0-9-\u0000-\u007F\s/()\-\_]{3,})\s*$"
)

def extract_headings(text: str) -> List[str]:
    heads, lines = [], text.splitlines()
    for ln in lines:
        s = ln.strip()
        if not s:
            continue
        if s.startswith("#"):
            heads.append(s)
        # if not markdown format
    if not heads:
        for ln in lines:
            m = HEADING_RE.match(ln)
            if m:
                heads.append(m.group(2).strip())
    return unique_ordered([re.sub(r"\s+", " ", h).strip() for h in heads])

from typing import List, Tuple, Dict

def heading_coverage(
    headings: List[str], required: List[str]
) -> Tuple[float, Dict[str, bool]]:
    found = {}
    norm = [h.lower() for h in headings]

    for req in required:
        if any(h.strip().startswith("##") for h in headings):
            found[req] = True
        else:
            found[req] = any(req.lower() in h for h in norm)

    score = sum(found.values()) / (len(required) or 1)
    return score, found

def heading_order_score(headings: List[str], required: List[str]) -> float:
    idx = {r: None for r in required}
    norm = [h.lower() for h in headings]
    for r in required:
        for i, h in enumerate(norm):
            if r.lower() in h:
                idx[r] = i
                break
    seq = [idx[r] for r in required if idx[r] is not None]
    if len(seq) <= 1:
        return 1.0 if seq else 0.0
    in_order = sum(1 for i in range(len(seq) - 1) if seq[i] <= seq[i + 1])
    return in_order / (len(seq) - 1)

```

ภาพที่ 16

จากภาพที่ 16 โค้ดนี้เกี่ยวกับการ ตรวจสอบโครงสร้างและหัวข้อ (headings) ของข้อความ โดยประกอบไปด้วย 3 ฟังก์ชันหลัก

1. `extract_headings(text: str) → List[str]`

โดยมีหน้าที่ในการดึงหัวข้อจากข้อความ โดยรองรับทั้ง Markdown (เช่น # Heading) และ รูปแบบทั่วไป (เช่น 1. บทนำ, - สรุป)

วิธีทำงาน: แยกข้อความเป็นบรรทัด ถ้าพบ # ที่ขึ้นต้น จะถือว่าเป็น heading หากถ้าไม่พบจะใช้ regex `HEADING_RE` ตรวจสอบบรรทัดที่มีลักษณะเหมือนหัวข้อ และคืนค่าเป็นรายการหัวข้อที่ไม่ซ้ำและเรียงตามลำดับ

2. `heading_coverage(headings, required) → (score, found)`

โดยมีหน้าที่ในการตรวจสอบว่า หัวข้อที่ต้องการ (required) มีอยู่ในรายการหัวข้อจริงหรือไม่ โดยมีหน้าที่ในการสร้าง dict found บอกว่าแต่ละหัวข้อ required พบหรือไม่ ถ้ามี Markdown แบบ ## ถือว่าพบ ถ้าไม่ ตรวจสอบว่ามีคำที่ต้องการอยู่ใน heading ผลลัพธ์ที่ได้: score มีค่าระหว่าง 0.0 ถึง 1.0 (ยิ่งสูงยิ่งครบ)

$$\text{score} = \frac{\text{จำนวนหัวข้อที่พบ}}{\text{จำนวนหัวข้อที่ต้องการ}}$$

3. `heading_order_score(headings, required) → float`

โดยมีหน้าที่ในการตรวจสอบว่า ลำดับหัวข้อ ตรงตามที่ต้องการหรือไม่

วิธีทำงาน: หา index ของแต่ละ required heading ในรายการจริง ถ้ามีแค่ 0 หรือ 1 heading คืนค่า 1.0 ถ้ามีหลาย heading จะตรวจสอบว่าลำดับเรียงถูกต้องกี่คู่ ผลลัพธ์ที่ได้: ค่าระหว่าง 0.0 ถึง 1.0 (ยิ่งสูงยิ่งเรียงถูกต้อง)

$$\text{score} = \frac{\text{จำนวนคู่ที่เรียงถูกต้อง}}{\text{จำนวนคู่ทั้งหมด}}$$

ใช้สำหรับประเมิน คุณภาพโครงสร้างเอกสาร เช่น รายงาน, บทความ, คู่มือ

ใช้ใน การตรวจสอบมาตรฐาน เช่น ต้องมีหัวข้อ "Introduction", "Method", "Conclusion"

ใช้ใน การทำ NLP หรือ Content Analysis เพื่อวิเคราะห์ความครบถ้วนและลำดับของเนื้อหา

```
# ----- coverage vs checklist -----
def coverage_score(text: str, checklist: List[str]) -> Tuple[float, Dict[str, float]]:
    toks = tokenize(text)
    joined = " ".join(toks)
    item_scores = {}
    for item in checklist:
        subs = [s.strip() for s in re.split(r"[;,]", item) if s.strip()] or [item]
        sub_hits = 0
        for s in subs:
            words = tokenize(s)
            if words:
                hit = sum(
                    1 for w in words if re.search(r"\b" + re.escape(w) + r"\b", joined)
                )
                sub_hits += 1 if hit >= max(1, math.ceil(len(words) * 0.6)) else 0
        item_scores[item] = sub_hits / (len(subs) or 1)
    score = sum(item_scores.values()) / (len(checklist) or 1)
    return score, item_scores
```

ภาพที่ 17

จากภาพที่ 17 โค้ดในภาพนี้เป็นฟังก์ชัน `coverage_score` ซึ่งใช้เพื่อวัดว่า ข้อความครอบคลุม checklist ที่กำหนดมากน้อยแค่ไหน หน้าที่ของฟังก์ชันคือการ รับ ข้อความ (text) และ รายการ checklist (checklist) ตรวจสอบว่าแต่ละ item ใน checklist ปรากฏในข้อความหรือไม่

การทำงาน ทำการเรียกฟังก์ชัน `Tokenize` ข้อความ เพื่อ แยกเป็นคำ ทำรวมคำทั้งหมดเป็น สตริงเดียว (joined) ดึงค่าแต่ละ item ของ checklist ที่เตรียมไว้ออกมา โดยแยก item เป็น `sub-items` โดยใช้ ; หรือ , เป็นตัวแบ่ง สำหรับแต่ละ sub-item `Tokenize` sub-item เพื่อให้ได้คำสำคัญ และตรวจสอบว่ามีคำเหล่านี้ในข้อความหรือไม่ ถ้าพบ $\geq 60\%$ ของคำใน sub-item นับว่า hit และนำค่าที่ได้มาคำนวณผล

$$\text{item score} = \frac{\text{จำนวน subitems ที่ hit}}{\text{จำนวน subitems}}$$

จากสูตรข้างบน ค่าที่ได้นั้นจะอยู่ในช่วง 0.0 ถึง 1.0 ยิ่งค่ามากยิ่งหมายความว่ายิ่งครอบคลุม โดยค่า 1 จะหมายถึง ครอบคลุมครบทุก item


```
# ----- redundancy -----
def repetition_score(report: str) -> float:
    toks = tokenize(report)
    sh = shingles(toks, n=6)
    if not sh:
        return 0.0
    total = sum(sh.values())
    repeated = sum(c for c in sh.values() if c > 1)
    return repeated / total if total else 0.0 # lower is better
```

ภาพที่ 18

จากภาพที่ 18 แสดงถึงฟังก์ชัน `repetition_score(report: str) -> float` ใช้เพื่อวัด ความซ้ำซ้อนของเนื้อหาในรายงาน โดยมีรายละเอียดดังนี้

หน้าที่ของฟังก์ชันคือตรวจสอบว่ามีการ ซ้ำของกลุ่มคำ (shingles) ในข้อความมากน้อยแค่ไหน คำนวณเป็น สัดส่วนของกลุ่มคำที่ซ้ำ (ยิ่งต่ำยิ่งดี)

วิธีทำงานคือทำการ Tokenize ข้อความแล้วแยกเป็นคำ (toks) สร้าง shingles ขนาด $n=6$ → คือกลุ่มคำต่อเนื่อง 6 คำ (ใช้ตรวจซ้ำ) ถ้าไม่มี shingles จะคืนค่า 0.0 และนำค่าที่ได้มาคำนวณผล

$$\text{score} = \frac{\text{จำนวนกลุ่มคำที่ปรากฏมากกว่า 1 ครั้ง}}{\text{จำนวนกลุ่มคำทั้งหมด}}$$

จากสูตรข้างบน ค่าที่ได้นั้นสามารถตีความหมายได้ว่ายิ่งถ้า score สูง แปลว่ามีการซ้ำเยอะ นั่นก็คือ ถ้าได้ค่า 0.0 หมายถึง ไม่มีการซ้ำเลย (ดีมาก), 0.1 – 0.3 → มีซ้ำเล็กน้อย (ปกติ) และถ้าคะแนน > 0.5 → ซ้ำเยอะ (ควรปรับปรุง) เนื่องจากมีเนื้อหาที่ถูกกล่าวซ้ำเกิดขึ้นเยอะ

```
# ----- units consistency -----
UNIT_RE = re.compile(
    r"\b(\d+(?:\.\d+)?) (\s?) (%|km|m|cm|mm|kg|g|thb|usd|baht|บาท|ครั้ง|คน|ปี|วัน|ชั่วโมง|นาที)\b",
    re.IGNORECASE,
)

def unit_consistency(report: str) -> Dict[str, Any]:
    entries = [(m.group(1), m.group(3).lower()) for m in UNIT_RE.finditer(report)]
    units = [u for _, u in entries]
    unit_counts = Counter(units)
    mixed_length = any(
        u in unit_counts and v in unit_counts
        for u in ["km", "m", "cm", "mm"]
        for v in ["km", "m", "cm", "mm"]
        if u != v
    )
    mixed_currency = any(
        u in unit_counts and v in unit_counts
        for u in ["thb", "baht", "บาท", "usd"]
        for v in ["thb", "baht", "บาท", "usd"]
        if u != v
    )
    return {
        "detected_pairs": entries,
        "unit_distribution": unit_counts,
        "potential_mixed_length_units": mixed_length,
        "potential_mixed_currency": mixed_currency,
    }
```

ภาพที่ 19

จากภาพที่ 19 ฟังก์ชัน `unit_consistency(report: str) -> Dict[str, Any]` ใช้เพื่อตรวจสอบว่าในข้อความมีการใช้ หน่วยวัด (units) สอดคล้องกันหรือไม่ โดยมีรายละเอียดดังนี้

หน้าที่ของฟังก์ชันคือการตรวจจับตัวเลขที่ตามด้วยหน่วย เช่น %, km, m, kg, บาท, usd, วัน, ชั่วโมง และนำมาวิเคราะห์ว่ามีการใช้หน่วยปนกันหรือไม่ เช่น km กับ m ในรายงานเดียวกัน

คืนค่าเป็น dictionary ที่บอก:

- detected_pairs → คู่ (ตัวเลข, หน่วย) ที่พบ
- unit_distribution → จำนวนแต่ละหน่วย
- potential_mixed_length_units → True ถ้ามีหลายหน่วยความยาว (เช่น km และ cm)
- potential_mixed_currency → True ถ้ามีหลายหน่วยเงิน (เช่น THB และ USD)

วิธีทำงานคือ ใช้ regex UNIT_RE หา pattern ตัวเลข (เช่น 12, 3.5) + ช่องว่าง (optional) + หน่วย (เช่น km, m, %, บาท) เพื่อสร้างรายการ entries ของคู่ (ตัวเลข, หน่วย) นับจำนวนแต่ละหน่วย ด้วย Counter และนำไปตรวจสอบ หากมีมากกว่า 1 หน่วยในกลุ่มความยาว mixed_length = True มีมากกว่า 1 หน่วยในกลุ่มเงิน → mixed_currency = True และจะคืนค่า dictionary ที่สรุปผลช่วงค่าที่ได้ ไม่ใช่ตัวเลข แต่เป็น dict เช่น

```
Python{ "detected_pairs": [("12", "km"), ("500", "m"), ("100", "thb")],
"unit_distribution": {"km": 1, "m": 1, "thb": 1}, "potential_mixed_length_units": True,
"potential_mixed_currency": False}Show more lines
```

ใช้สำหรับตรวจสอบ ความสอดคล้องของหน่วย ในรายงาน เช่น ไม่ควรใช้ km และ m ปนกันโดย ไม่ชี้แจง ไม่ควรใช้ THB และ USD ปนกันโดยไม่ระบุอัตราแลกเปลี่ยน ช่งยใน การทำ Data Quality Check, การตรวจสอบเอกสารทางเทคนิค, การทำ NLP สำหรับเอกสาร

```
# ----- faithfulness (ถ้ามี sources) -----
def sentence_support(sentence: str, sources: List[str], n: int = 5) -> float:
    sh = set(shingles(tokenize(sentence), n).keys())
    if not sh:
        return 0.0
    best = 0.0
    for src in sources:
        sc = jaccard(sh, set(shingles(tokenize(src), n).keys()))
        if sc > best:
            best = sc
    return best

def faithfulness(report: str, sources: Optional[List[str]]) -> Dict[str, Any]:
    if not sources:
        return {"support_ratio": None, "per_sentence": []}
    sents = split_sentences(report)
    scores = [sentence_support(s, sources, n=5) for s in sents]
    supported = sum(1 for sc in scores if sc >= 0.12) # threshold แบบหยวน
    return {
        "support_ratio": round(supported / (len(sents) or 1), 3),
        "per_sentence": [
            {"sentence": s, "support": round(sc, 3)} for s, sc in zip(sents, scores)
        ],
    }
```

ภาพที่ 20

จากภาพที่ 20 เป็นการวัดผลค่า ความสอดคล้องข้อมูลจากสองส่วนคือ

ส่วนที่ 1 `sentence_support(sentence, sources, n=5) → float` เป็นการวัดว่า ประโยคหนึ่งในรายงานมีการสนับสนุนจากแหล่งข้อมูลมากน้อยแค่ไหน โดยใช้ Jaccard similarity ระหว่างชุด shingles (กลุ่มคำต่อเนื่อง) ของประโยคและแหล่งข้อมูล วิธีทำงานคือ แปลงประโยคเป็น tokens แล้วสร้าง shingles ขนาด n=5 ถ้าไม่มี shingles ให้คืนค่า 0.0 เปรียบเทียบกับแต่ละ source โดยใช้ Jaccard index คืนค่า similarity ที่ดีที่สุด

ส่วนที่ 2 `faithfulness(report, sources) → Dict[str, Any]` มีหน้าที่วัดความสอดคล้องของทั้งรายงานกับแหล่งข้อมูล โดย แบ่งรายงานเป็นประโยค คำนวณ `sentence_support` สำหรับแต่ละประโยค นับว่ามีกี่ประโยคที่มี `score ≥ 0.12` (threshold แบบหยาบ) และคืนค่า `support_ratio` คือ สัดส่วนของประโยคที่ได้รับการสนับสนุนและค่า `per_sentence` → รายละเอียดแต่ละประโยคและคะแนน

ช่วงค่าที่ได้ถ้า `support_ratio = None` คือไม่มี sources ให้ตรวจสอบ โดยค่าจะอยู่ในช่วง 0.0 ถึง 1.0 คือยิ่งสูงยิ่งสอดคล้องกับแหล่งข้อมูลนั่นเอง

ใช้สำหรับ ตรวจสอบว่า รายงานอ้างอิงข้อมูลจริง หรือไม่ ใช้ใน การประเมินคุณภาพสรุป (summary evaluation) เช่น ในงาน NLPs หรือ ใช้ใน การตรวจสอบ fact-checking สำหรับเนื้อหาที่ต้องอ้างอิงแหล่งข้อมูล

```
weights = {
    "coverage": 0.25,
    "structure": 0.35,
    "clarity": 0.15,
    "non_redundancy": 0.15,
    "faithfulness": 0.10,
}

overall = (
    weights["coverage"] * cov
    + weights["structure"] * structure_component
    + weights["clarity"] * fk_to_score(fk)
    + weights["non_redundancy"] * (1.0 - rep)
    + weights["faithfulness"] * faith_component
)
```

ภาพที่ 21

จากภาพที่ 21 เป็นสูตรการคำนวณค่าผลรวมที่เกิดจากค่าเฉลี่ยแบบถ่วงน้ำหนัก โดยอาศัยองค์ประกอบหลักดังนี้ องค์ประกอบที่ใช้

1. **coverage (cov)** คือ คะแนนความครอบคลุม checklist
น้ำหนัก: 0.25

2. **structure_component** คือ คะแนนโครงสร้างหัวข้อ

คำนวณจาก: $\text{structure_component} = 0.6 * \text{head_cov} + 0.4 * \text{head_order}$

- head_cov = ความครบของหัวข้อ
- head_order = ความถูกต้องของลำดับหัวข้อ

น้ำหนัก: 0.35

3. **clarity** คือ คะแนนความชัดเจนของภาษา

แปลงจาก Flesch-Kincaid grade (fk) ด้วยฟังก์ชัน: $\text{fk_to_score}(\text{grade}) = 1.0 - \text{abs}(\text{grade} - 9.5) / 7.5$ (ยิ่งใกล้ระดับอ่านง่ายสำหรับคนทั่วไปจะคะแนนสูง)

น้ำหนัก: 0.15

4. **non_redundancy** คือ ความไม่ซ้ำซ้อน

ใช้ $(1.0 - \text{rep})$ โดย rep = repetition score (ยิ่งซ้ำมากจะคะแนนต่ำ)

น้ำหนัก: 0.15

5. **faithfulness** → ความสอดคล้องกับแหล่งข้อมูล

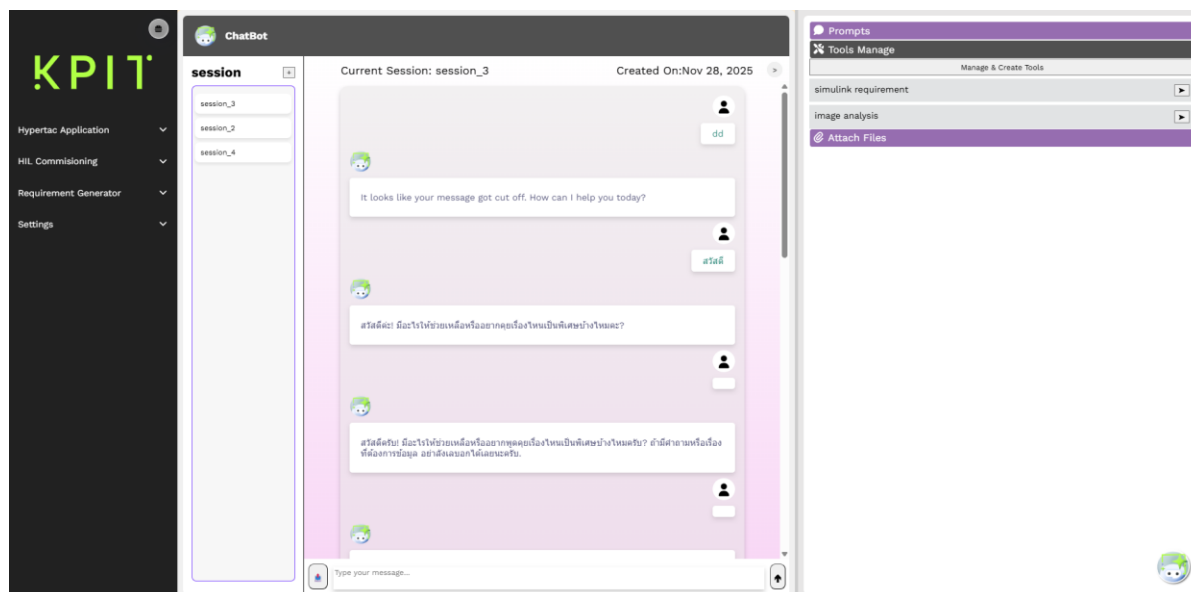
ใช้ `faith["support_ratio"]` ถ้ามี sources, ถ้าไม่มีจะใช้ 0.5

น้ำหนัก: 0.10

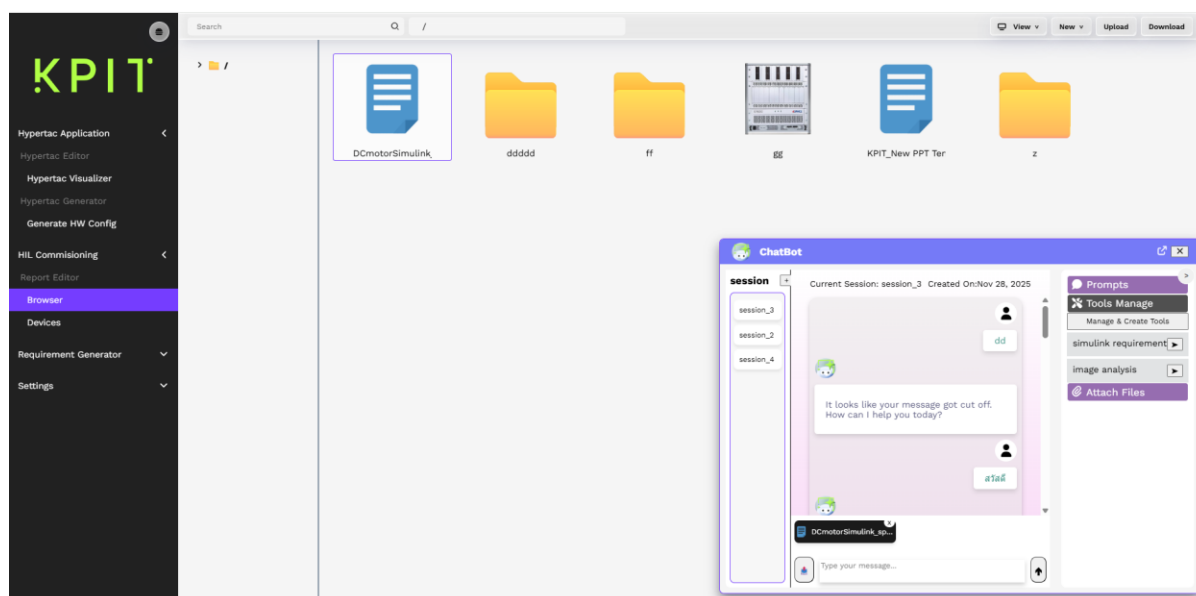
4.1.4 ฟีเจอร์ส่งออกเอกสารแบบมีอาชีพ

แอปพลิเคชันช่วยให้วิศวกรสามารถส่งออกข้อกำหนดที่สร้างขึ้นเป็นเอกสารที่จัดรูปแบบอย่างมืออาชีพ ฟีเจอร์ “Share as Word” และ “Share as PDF” ถูกพัฒนาอย่างละเอียดเพื่อรองรับการจัดรูปแบบที่ซับซ้อน เช่น ตาราง หัวข้อ ฟอนต์ และข้อความตัวหนา/ตัวเอียง เพื่อให้ผลลัพธ์พร้อมใช้งานโดยช่วยลดการแก้ไขเพิ่มเติม

4.1.5 Chatbot Interface สำหรับโต้ตอบ



รูปภาพที่ 22 Chat Bot



รูปภาพที่ 23 Chatbot Mini

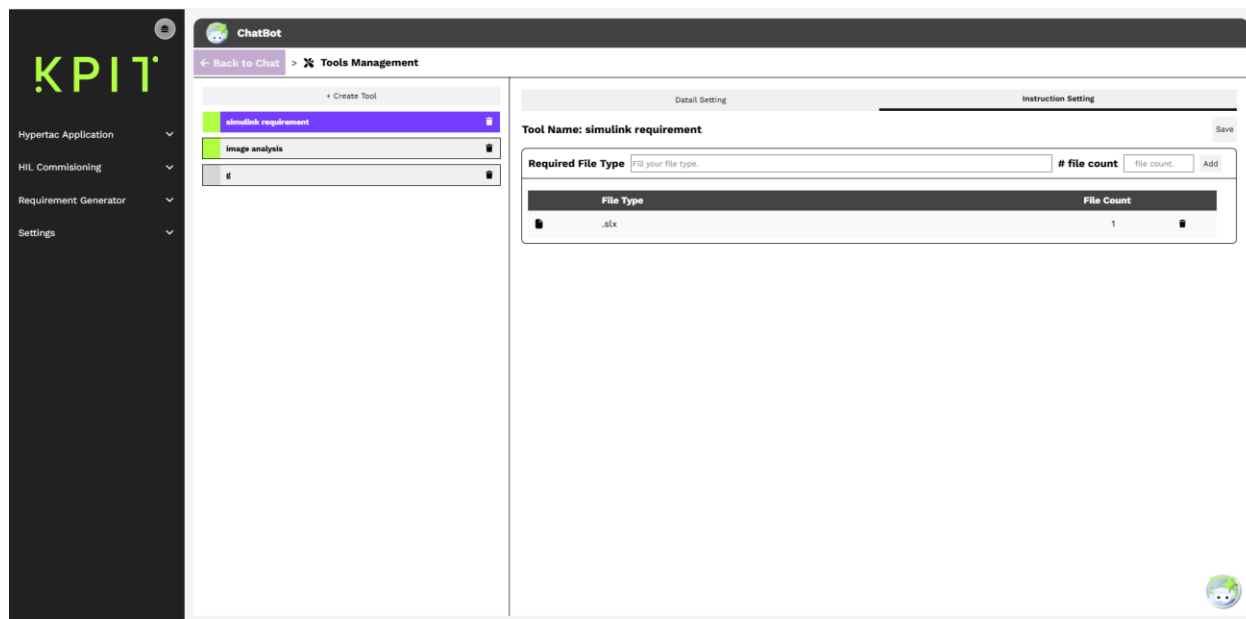
จากภาพที่ 22 และ 23 พัฒนาแชตบอทที่ใช้งานง่ายเป็นอินเทอร์เฟซหลัก เพื่อลดความซับซ้อนของกระบวนการที่อยู่เบื้องหลัง โมเดลการสนทนานี้ช่วยให้ผู้ใช้งานสามารถสร้างข้อกำหนดด้วยการถามตอบแบบภาษาธรรมชาติ แทนที่จะต้องใช้เมนูที่ซับซ้อน พร้อมฟีเจอร์เสริม เช่น การจัดการหลายเซสชัน ประวัติการสนทนา และการอัปโหลดไฟล์แบบลากและวาง เพื่อเพิ่มประสิทธิภาพการทำงานและการยอมรับของผู้ใช้

4.1.6 ระบบจัดการไฟล์และโฟลเดอร์

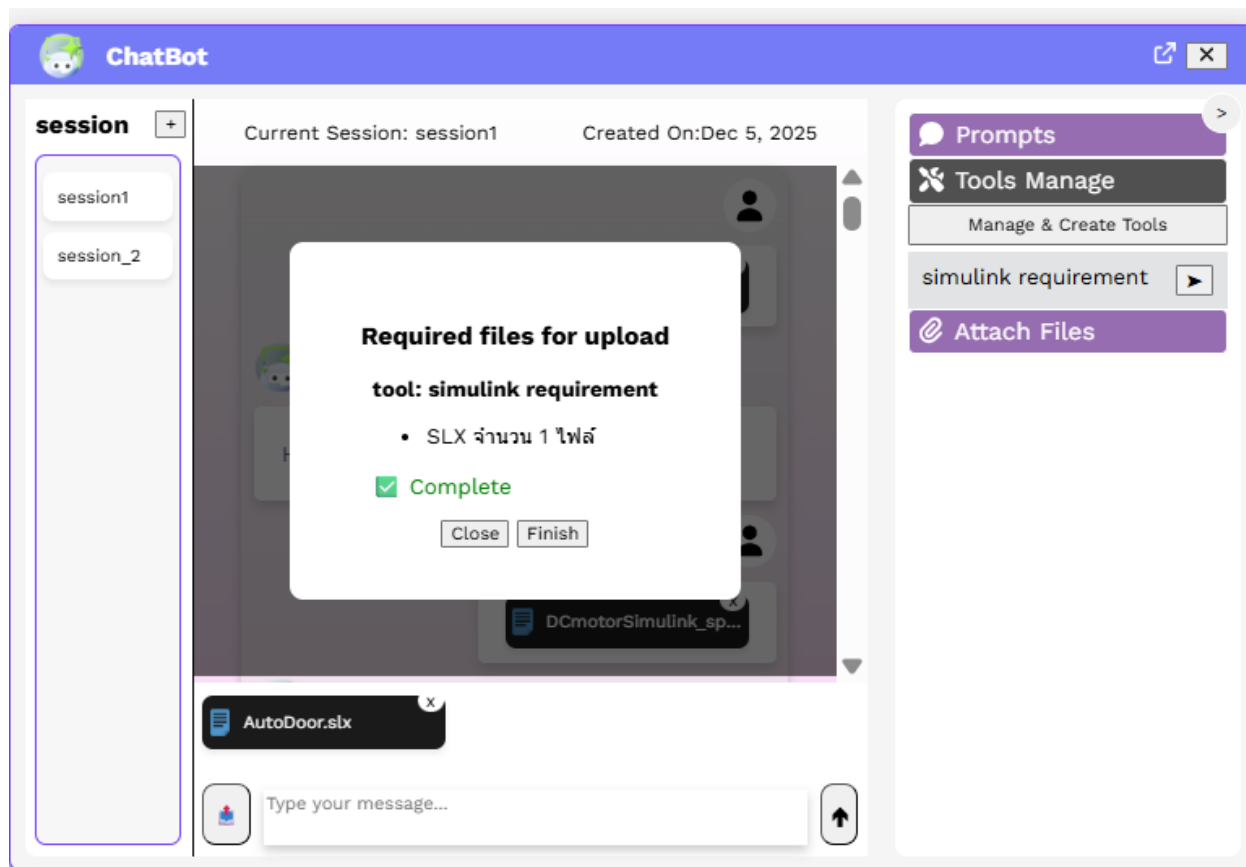
เว็บไซต์ที่มีระบบจัดการไฟล์และโฟลเดอร์ที่สมบูรณ์ รองรับการทำงาน CRUD (Create, Read, Update, Delete) เพื่อให้ผู้ใช้สามารถจัดระเบียบโครงการ อัปโหลดไฟล์ใหม่ และจัดการพื้นที่ทำงานได้โดยตรงในแอปพลิเคชัน ลดความจำเป็นในการใช้เครื่องมือจัดการไฟล์ภายนอก ทำให้ประสบการณ์ใช้งานราบรื่นขึ้น ซึ่งภายหลังยกเลิกและไปรวมกับกับระบบจัดการไฟล์และโฟลเดอร์ที่มีอยู่ในอีกโปรเจก

4.1.7 ระบบจัดการและเลือกเครื่องมือ (Tool Management)

รูปภาพที่ 24 ระบบ Tool Management Detail Setting



รูปภาพที่ 25 ระบบ Tool Management Instruction Setting



รูปภาพที่ 26 ภาพการใช้งานร่วมกับ Chatbot

จากรูปที่ 17,18 และ 19 เป็นระบบต้นแบบสำหรับจัดการเครื่องมือที่ให้ผู้ใช้งานกำหนด ตั้งค่าผ่าน User Interface โดยเครื่องมือเหล่านี้สามารถส่งไปยัง API ฟังก์ชัน Backend เพื่อเลือกเครื่องมือที่มีอยู่จากการจับ Keyword และ ชื่อของเครื่องมือ เพื่อเลือกฟังก์ชันการทำงานของแต่ละ Tool เป็นการพัฒนาเพื่อลดการขยายเครื่องมือของ Chat Bot ในอนาคตในการจัดการไฟล์ .slx โดยระบบนี้จะสามารถเลือกได้ว่าจะให้ Tool ไหน Active บ้างเพื่อป้องกันการไม่ให้เกิดการจับคู่ในทุกๆคำถามที่ส่งไปยัง Chatbot เป็นการช่วยลดความผิดพลาดของระบบ รวมทั้งมีการสร้าง File Instruction สำหรับกำหนดเงื่อนไขการรับไฟล์แต่ละ Tool จำนวน File ที่จะใช้ในการประมวลผล

4.1.8 ระบบยืนยันตัวตนด้วย API Token ที่ปลอดภัย

เพื่อความปลอดภัยในการสื่อสารกับบริการแบ็กเอนด์และ API ของ KGPT ได้มีการใช้ระบบยืนยันตัวตนแบบ Token รวมถึงการตรวจสอบ Token และการจัดการ Cookie อย่างปลอดภัย เพื่อป้องกันการเชื่อมต่อที่สำคัญและตรวจสอบสิทธิ์ผู้ใช้งานก่อนประมวลผลคำขอ ซึ่งเป็นชั้นความปลอดภัยที่จำเป็นสำหรับแอปพลิเคชันระดับองค์กร

4.2 ผลการปฏิบัติงานประจำ

ส่วนสำคัญของการฝึกงานคือการมีส่วนร่วมกับทีมและสภาพแวดล้อมของบริษัท ซึ่งรวมถึงการเข้าร่วมกระบวนการทำงานร่วมกัน การแบ่งปันความรู้ และการปฏิบัติตามแนวทางวิชาชีพ

4.2.1 เรียนรู้การพัฒนาโปรเจกต์ร่วมกับผู้อื่น

การทำงานร่วมกันในโครงการซอฟต์แวร์เป็นทักษะสำคัญที่ช่วยให้ทีมสามารถพัฒนาโค้ดได้อย่างมีประสิทธิภาพ โดยในโครงการนี้ได้เรียนรู้และปฏิบัติจริงผ่านการใช้ Git Version Control บน GitLab ซึ่งเป็นแพลตฟอร์มสำหรับการจัดการซอร์สโค้ดและการทำงานร่วมกันของทีมพัฒนา โดยได้เรียนรู้การใช้ Git Version Control อย่างถูกต้อง ทำความเข้าใจแนวคิดของ Version Control เพื่อจัดการการเปลี่ยนแปลงของโค้ด ใช้คำสั่งพื้นฐาน เช่น git clone, git add, git commit, git push, git pull เพื่อซิงค์งานกับทีม รวมถึงการใช้ Commit Message Format ปฏิบัติตามมาตรฐานการเขียน commit message เช่น

- feat: เพิ่มฟีเจอร์ใหม่สำหรับการอัปโหลดไฟล์
- fix: แก้ไขบั๊กในระบบจัดการโฟลเดอร์
- refactor: ปรับปรุงโครงสร้างโค้ดเพื่อความอ่านง่าย

การใช้ format ที่ชัดเจนช่วยให้ทีมเข้าใจการเปลี่ยนแปลงได้ง่าย และรองรับการสร้าง Changelog อัตโนมัติในอนาคตการแตก Branch เพื่อทำงานแบบขนาน ซึ่งการใช้ branch แยกงานช่วยลดความเสี่ยงในการเกิด conflict และทำให้การรวมโค้ดเป็นระบบมากขึ้น การแก้ไข Conflict และการ Merge

4.2.2 การแบ่งปันความรู้

มีส่วนร่วมในการเพิ่มพูนความรู้ของทีมโดยจัดทำและนำเสนอเกี่ยวกับ File Hub API นอกจากนี้ยังเข้าร่วมการแลกเปลี่ยนความรู้ภายในในหัวข้อ เช่น TypeScript เทคนิคการพัฒนาฐานข้อมูล Docker และ Svelte เพื่อเรียนรู้อย่างต่อเนื่องจากเพื่อนร่วมงาน ความรู้ที่ได้รับจากการประชุมเหล่านี้ช่วยสนับสนุนการปรับปรุงโค้ดเบสของโครงการให้ทันสมัย

4.2.3 การทำงานร่วมกับทีม

เข้าร่วมประชุมอัปเดตงานประจำสัปดาห์และประชุมโครงการอย่างสม่ำเสมอ พร้อมรายงานความคืบหน้า และร่วมอภิปรายเพื่อให้สอดคล้องกับเป้าหมายความต้องการ

4.2.4 การสื่อสารโดยใช้ภาษาอังกฤษ

ได้รับประสบการณ์ในการสื่อสารด้วยภาษาอังกฤษเพื่อนำเสนอข้อมูลอัปเดตเกี่ยวกับ File Hub API ให้กับทีมในต่างประเทศ

4.2.5 การเรียนรู้จากเพื่อนร่วมงาน

ได้รับประโยชน์จากวัฒนธรรมการทำงานร่วมกันของบริษัทโดยเข้าร่วมการถ่ายทอดความรู้จากเพื่อนฝึกงานคนอื่น แสดงถึงความตั้งใจที่จะเรียนรู้จากผู้อื่น

5. บทสรุป

5.1 สรุปผลและการนำไปใช้ประโยชน์

การฝึกงานครั้งนี้สิ้นสุดลงด้วยการพัฒนา Proof-of-Concept ที่ทำงานได้จริงสำหรับเครื่องมือสร้าง Requirement ด้วย AI โครงการบรรลุเป้าหมายหลัก สร้างระบบที่สามารถรับไฟล์ .slx ประมวลผลโดยใช้โมเดล LLM ของบริษัท และแสดงผลผ่าน Chatbot และ Browse บนเว็บ

ประโยชน์หลักและผลกระทบที่อาจเกิดขึ้น

- **การทำงานอัตโนมัติ** เครื่องมือนี้ช่วยแก้ปัญหการสร้างเอกสารข้อกำหนดจากโมเดล Simulink ที่ซับซ้อนแบบแมนนวล โดยการทำงานอัตโนมัติสามารถประหยัดเวลาในการสร้างเอกสาร Software Requirement ช่วยลดความผิดพลาด และให้วิศวกรมีเวลามากขึ้นสำหรับงานออกแบบและวิเคราะห์ที่มีมูลค่าสูง
- **การใช้งานที่ง่ายขึ้น** แดชบอร์ดที่ใช้งานง่ายและระบบจัดการไฟล์แบบบูรณาการช่วยให้ผู้ใช้เข้าถึงกระบวนการที่ซับซ้อนได้ง่ายขึ้น ส่งเสริมการใช้งานโดยทำให้ความสามารถของ AI ที่ทรงพลังใช้งานได้สะดวก
- **ความสามารถในการขยาย** แอปพลิเคชันถูกสร้างบนเทคโนโลยีสมัยใหม่ที่มีโครงสร้างแบบโมดูล (Svelte, Node.js, MySQL) ซึ่งเป็นรากฐานที่มั่นคงและสามารถขยายได้ในอนาคต เช่น เพิ่มฟีเจอร์ใหม่ รองรับไฟล์ประเภทอื่น และเชื่อมต่อกับระบบองค์กรอื่น รวมทั้งระบบ Tool Management ที่รองรับการขยายเครื่องมือเกี่ยวกับ AI ในอนาคต

5.2 ปัญหาที่พบระหว่างการปฏิบัติงาน และการแก้ปัญหา

การแก้ปัญหาเป็นทักษะสำคัญในงานพัฒนาซอฟต์แวร์ ตลอดโครงการมีความท้าทายทางเทคนิคหลายประการที่ต้องวิเคราะห์ แก้ไข และปรับกลยุทธ์อย่างรอบคอบ โดยอุปสรรคสำคัญและวิธีแก้ไขที่นำมาใช้มีดังนี้

5.2.1 ข้อจำกัดของการพัฒนาร่วมกับ LLM ในบริษัทจาก Token ที่มีจำกัดและไม่สามารถทำการ Fine Tuning กับ LLM ได้โดยตรง โดยแนวทางแก้ปัญหานำมาใช้ได้แก่

5.2.1.1 การจัดการปัญหา Token จำกัด

- ใช้ Text Splitters + Chunking

แบ่งข้อมูลเป็นส่วนเล็ก ๆ (chunks) และใช้ Retrieval-Augmented Generation (RAG) เพื่อดึงเฉพาะข้อมูลที่เกี่ยวข้องกับคำถาม

- ลดขนาด Prompt

จากการใช้ Prompt Engineering เพื่อลดคำที่ไม่จำเป็น และการใช้ Template ที่กระชับ และตัดข้อมูลซ้ำซ้อน

- ใช้ Embeddings + Vector Database

แทนที่จะส่งข้อมูลทั้งหมด ให้เก็บข้อมูลใน Vector DB (เช่น FAISS, Pinecone, Chroma) แล้วดึงเฉพาะส่วน Chunk ที่เกี่ยวข้อง

- เลือกโมเดลที่มี Context Window ใหญ่ขึ้น

ถ้าเป็นไปได้ ใช้โมเดลที่รองรับ token มากขึ้น เช่น GPT-4 Turbo หรือ LLM ที่มี 128k context

5.2.1.2 ไม่มี Fine-Tuning โดยตรง

- ใช้ Prompt Engineering ขั้นสูง

สร้าง Prompt ที่มีโครงสร้างชัดเจน (System + Human messages)

- ใช้ Few-Shot Learning โดยใส่ตัวอย่างใน Prompt

การใช้ RAG ร่วมกับ Knowledge Base เพิ่มข้อมูลเฉพาะของบริษัทใน Vector DB ทำให้โมเดลตอบตามข้อมูลจริง โดยไม่ต้องปรับโมเดล และใช้ Parameter-Efficient Tuning (ไม่ได้ลองรับ) เช่น LoRA หรือ Adapter (ถ้า API รองรับ) เพื่อปรับโมเดลบางส่วนโดยไม่ต้อง Fine-Tune เต็มรูปแบบ

- สร้าง Layer ของ Business Logic

ใช้ Chain หรือ Agent ใน Lang Chain เพื่อควบคุมการตอบทำ Post-Processing เพื่อปรับคำตอบให้ตรงตามมาตรฐานบริษัท

5.2.2 การทำงานของ RAG ร่วมกับ LLM ในตอนแรกไม่สามารถประมวลผลไฟล์ทั้งหมดได้ ทำให้บริบทไม่ครบถ้วน

ได้มีการทดลองแนวคิด RAG ไปประยุกต์ใช้โดยตรงกับไฟล์ .slx แต่พบปัญหาว่า LLM ไม่สามารถอ่านเนื้อหาจากไฟล์ทั้งหมดได้เนื่องจาก กระบวนการของ RAG ที่มีการ embedding คำถามทั้งหมดไปเป็นในรูปแบบ vector และนำไปทำการค้นหาแบบ Semantic เพื่อคำนวณค่าคะแนน Similarity ของแต่ละ Chunk ที่เก็บไว้และเลือกอันที่มีค่าคะแนนสูงสุดออกมาและแปลงออกมาเป็นภาษามนุษย์อีกที เลยทำให้เกิดการทกล่นของข้อมูลจากคำถามหนึ่งคำถามที่ต้องการคำตอบจากหลายส่วน ซึ่งการสร้างเอกสารนั้นมีแค่คำถามเดียวและไม่มีการเปลี่ยนแปลงของคำถาม กระบวนการ RAG ที่มีการ embedding เพิ่มขึ้นนั้นเลยไม่จำเป็น จึงได้มีการปรับกระบวนการทำใหม่ โดยเริ่มจากการแตกไฟล์ .slx แล้วดึงข้อมูล XML ที่จำเป็นออกมา ก่อนเพื่อนำข้อมูลที่มีโครงสร้างชัดเจนนี้รวมเข้ากับ prompt template ซึ่งช่วยให้ LLM ได้รับบริบทครบถ้วน และสร้าง Software Requirements ได้แม่นยำครบถ้วนตามโครงสร้างของรายงานมากขึ้น

5.2.3 ระบบจัดการไฟล์และโพลเดอร์มีบักจำนวนมาก เช่น การคำนวณ การแสดงผล และการอัปเดตฐานข้อมูลผิดพลาด

ดำเนินการดีบั๊กอย่างเป็นระบบ แยกแต่ละบัก ระบุสาเหตุ และแก้ไขพร้อมทดสอบเพื่อให้ระบบมีเสถียรภาพและความถูกต้องของข้อมูล

5.2.4 การสร้างไฟล์ Word ในตอนแรกทำบนฝั่ง Client (Frontend) ซึ่งไม่มีประสิทธิภาพและไม่สามารถขยายได้

แนวทางเดิมทำให้โหลดการประมวลผลอยู่บนเบราว์เซอร์ของผู้ใช้ จึงปรับไปใช้ API ฝั่ง Backend เพื่อรวม logic การสร้างเอกสาร ทำให้ระบบแข็งแรงและดูแลรักษาง่ายขึ้น

5.2.5 การรวมฟีเจอรืใหม่จากหลาย branch ทำให้เกิด merge conflict บ่อยครั้งใน GitLab

ใช้วิธีการควบคุมเวอร์ชันอย่างมีระบบ โดยอัปเดต branch อย่างระมัดระวัง สื่อสารกับทีม และแก้ไข conflict อย่างเป็นขั้นตอนเพื่อให้การรวมโค้ดสะอาดและเสถียร

ความท้าทายเหล่านี้ตอกย้ำหลักการสำคัญของวิศวกรรมซอฟต์แวร์ ได้แก่ การออกแบบสถาปัตยกรรมที่แข็งแรงตั้งแต่แรก ความจำเป็นของการดีบั๊กอย่างเป็นระบบ และบทบาทสำคัญของการควบคุมเวอร์ชันที่มีวินัยในการทำงานร่วมกัน การแก้ปัญหาเหล่านี้ไม่ใช่แค่การแก้บัก แต่เป็นการสร้างความยืดหยุ่นและความเข้าใจเชิงลึกในการพัฒนาระบบจริง

5.3 ข้อเสนอแนะ

แม้โครงการจะบรรลุเป้าหมายหลักและสร้าง Proof-of-Concept ที่แข็งแกร่ง แต่ยังมีโอกาสในการปรับปรุงและขยายเพื่อเพิ่มคุณค่าและประโยชน์ ดังนี้:

5.3.1 พัฒนาระบบประเมินผลให้สมบูรณ์

สำหรับระบบการสร้าง Software Requirement นั้นเนื่องจากมีตัวอย่างเพียงไฟล์เดียวจึงไม่สามารถวัดค่าประสิทธิภาพอย่างค่าความแม่นยำได้โดยตรง สามารถวัดผลได้แค่ โครงสร้างข้อมูลทั่วไป การใช้คนที่มีทักษะและความรู้มาตรวจสอบอีกครั้งในทุกๆ ครั้งจะช่วยให้เอกสารสมบูรณ์และถูกต้องมากขึ้น และอาจตัดพหุค่าที่ไม่จำเป็นหรือปรับลดความสำคัญจากค่าน้ำหนักลง เช่นค่า ความอ่านง่าย (Readability) เนื่องจากเอกสารมี code logic ประกอบอยู่ด้วยการประเมินค่านี้จึงมักได้ค่าที่สูง ซึ่งไม่ได้ส่งผลอะไร

และจาก Tool Management ยังขาดในส่วนของการวัดผลที่แต่ละเครื่องมือ ซึ่งแต่ละเครื่องมือนั้นก็มีการวัดผลที่แตกต่างออกไป ควรพัฒนาไปสู่ระบบอัตโนมัติเต็มรูปแบบเพื่อเชื่อมเข้ากับการวัดผลที่สามารถทำได้หลากหลายรูปแบบ เพื่อให้ผู้ใช้ได้รับผลการประเมินเชิงปริมาณทันที และใช้ติดตามการปรับปรุงประสิทธิภาพของ LLM ในระยะยาว

5.3.2 รองรับไฟล์ประเภทอื่น

ปรับ logic ฝั่ง Backend ให้รองรับไฟล์วิศวกรรมประเภทอื่นนอกจาก .slx เพื่อเพิ่มการใช้งานในสาขาวิศวกรรมที่หลากหลายและขยายฐานผู้ใช้

5.3.3 สร้างกระบวนการรับข้อเสนอแนะจากผู้ใช้ (Feedback Loop)

ดำเนินการทดสอบการยอมรับของผู้ใช้ (UAT) กับกลุ่มผู้ใช้เป้าหมาย เพื่อเก็บข้อมูลเชิงโครงสร้างเกี่ยวกับการใช้งานแพลตฟอร์ม ฟีเจอร์ และประสิทธิภาพ เพื่อปรับปรุง UI/UX และจัดลำดับความสำคัญของการพัฒนาในอนาคต

5.3.4 ปรับปรุงโค้ดเบสให้ทันสมัยต่อเนื่อง

ดำเนินการย้ายสคริปต์จาก Python ไปเป็น TypeScript/Node.js ต่อไป เพื่อสร้างโค้ดเบสที่เป็นหนึ่งเดียวดูแลรักษาง่าย และลดจุดเสี่ยงของระบบ

5.3.5 ปรับปรุงการออกแบบ Prompt สำหรับ LLM

สร้างกระบวนการทดสอบ A/B สำหรับเทมเพลต Prompt เพื่อทดลองและปรับปรุงคำสั่งที่ส่งไปยัง LLM อย่างต่อเนื่อง เพื่อเพิ่มความแม่นยำ ความชัดเจน และโครงสร้างของผลลัพธ์ที่ AI สร้างขึ้น

5.3.6 ปรับเปลี่ยน Model

เนื่องจากบริษัทมีการปรับเปลี่ยนและพัฒนา LLM Model ที่ให้บริการอย่างสม่ำเสมอ หากนำโมเดลใหม่มาปรับใช้อาจสามารถเพิ่มขีดจำกัดได้ด้านต่างๆได้ เช่น จำนวน token, ความสามารถในการรับออกไฟล์ได้หลากหลายประเภทมากขึ้น

5.3.7 การขยายเข้าสู่ FILE HUB

ทำระบบเชื่อมเข้าสู่ FILE HUB อัตโนมัติ โดยส่งเฉพาะไฟล์ที่ FILE HUB สามารถทำการ RAG ได้เพื่อใช้งานประโยชน์ในด้านความปลอดภัย การแชร์ไฟล์ กำหนดสิทธิการเข้าถึง ระบบล็อกอิน SSO ของบริษัท ทำให้เกิดการเชื่อมต่อที่มีประสิทธิภาพมากยิ่งขึ้นนั่นเอง

5.3.7 ข้อจำกัดของการรับ Token

หากข้อมูลจากโมเดล .slx มีขนาดใหญ่ ต้องระวังความลึกของโครงสร้างข้อมูล เนื่องจากการแบ่งข้อมูลเพื่อป้อนเข้าโมเดลที่ละกิ่ง (branch) อาจทำให้บางกิ่งมีขนาดใหญ่เกินขีดจำกัดของ Token ซึ่งจะส่งผลให้ไม่สามารถดำเนินการต่อได้ แนวทางการพัฒนาที่แนะนำคือการปรับปรุงวิธีการทำ Chunk ให้เหมาะสมกับข้อมูลที่มีโครงสร้างความสัมพันธ์แบบต้นไม้ เพราะวิธีการ Chunk แบบเดิมมักเหมาะกับข้อมูลที่เป็นข้อความยาวในรูปแบบย่อหน้า (paragraph) มากกว่า โดยหลังจากการ Chunk ควรนำข้อมูลเข้าสู่กระบวนการ Embedding และจัดเก็บในรูปแบบ Vector เพื่อให้เหมาะสมกับการค้นคืน (Retrieval) และการประมวลผลในขั้นตอนต่อไป

6. บรรณานุกรม

[1] AiPromptsX. (n.d.). Complete Guide to AI Prompt Frameworks: APE, RACE, ROSES & More.

Retrieved December 4, 2025, from <https://aipromptsx.com>

[2] Bhargav, S. (2025, n.d.). GenAI 101: AI, ML, LLMs Explained. Retrieved December 4, 2025, from

<https://medium.com/@sudhanshu.bhargav/genai-101-ai-ml-llms-explained-2e324aa2a0b0>

[3] docx.js. (n.d.). Documentation. Retrieved December 4, 2025, from <https://docx.js.org>

[4] KGPT API Documentation. (n.d.). KGPT API Documentation. Retrieved December 4, 2025, from

<https://kgpt.ai/docs>

[5] LangChain. (n.d.). Home - Docs by LangChain. Retrieved December 4, 2025, from

<https://python.langchain.com/docs>

[6] Prompt Engineering Guide. (n.d.). Basics of Prompting. Retrieved December 4, 2025, from

<https://www.promptingguide.ai>

[7] Svelte. (n.d.). Svelte Documentation. Retrieved December 4, 2025, from <https://svelte.dev>

ภาคผนวก

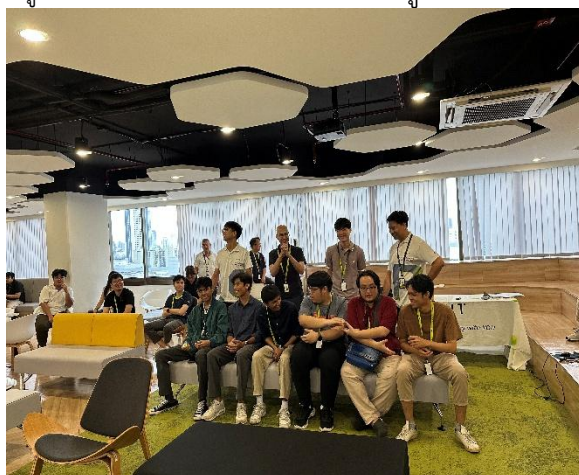
ประมวลผลภาพบรรยากาศการฝึกงาน



รูปภาพที่ 1 เข้าร่วมกิจกรรม CSR ปลูกล้าชายเลน



รูปภาพที่ 2 เข้าร่วมกิจกรรม CSR ปลูกล้าชายเลน



รูปภาพที่ 3 เข้าร่วมฟังกิจกรรม Tech Day



รูปภาพที่ 4 เข้าร่วมฟังกิจกรรม Tech Day



รูปภาพที่ 5 ส่งเพื่อนจบฝึกงาน



รูปภาพที่ 6 เข้าฟัง knowledge sharing



รูปภาพที่ 7 กินเลี้ยงส่งเพื่อนฝึกงาน



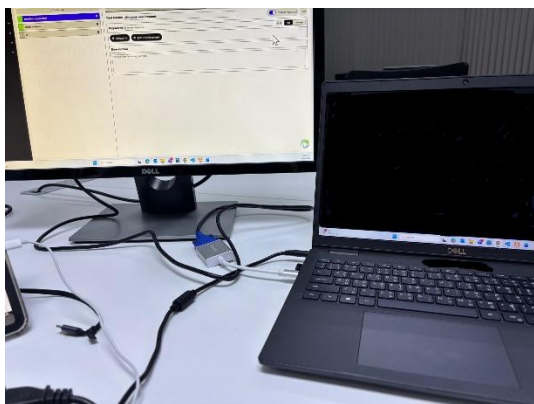
รูปภาพที่ 8 กินเลี้ยงวันเกิดเพื่อนฝึกงาน



รูปภาพที่ 9 กินเลี้ยงส่งเพื่อนฝึกงาน



รูปภาพที่ 10 กินเลี้ยงรับเข้าฝึกงาน



รูปภาพที่ 11 บรรยากาศการทำงาน



รูปภาพที่ 12 การนิเทศ